

“The Enormous Theorem”

Una sola dimostrazione.

10 000

pagine

500 articoli · oltre 100 autori · mezzo secolo

Nessuno l'ha letta tutta.



La matematica sta superando la scala delle menti umane

Teoria della dimostrazione, Teoria dei tipi, Proof Assistants e AI

verso una nuova pratica della matematica

Marino Miculan

Scuola Estiva di Logica AILA 2026

Bardonecchia, 31 agosto – 5 settembre 2026

DMIF, Università di Udine | marino.miculan@uniud.it | <https://marino.miculan.org>

La crisi della complessità

Perché un corso di teoria della dimostrazione, oggi

La crisi della complessità

La matematica contemporanea affronta un problema **senza precedenti**.

- ▶ dimostrazioni di centinaia — o migliaia — di pagine non sono più l'eccezione;
- ▶ teoremi che richiedono *anni* per essere letti e compresi, e forse mai fino in fondo;
- ▶ il collo di bottiglia della peer review: **chi verifica i verificatori?**

⚠ Il punto

Non è una difficoltà passeggera: è una questione di **scala**. Verificare una dimostrazione sta diventando un problema che eccede la capacità di una singola persona.



L'esempio paradigmatico: *The Enormous Theorem*

Teorema di classificazione dei gruppi semplici finiti: ogni gruppo semplice finito è ciclico di ordine primo, alterno, di tipo Lie, oppure uno di 26 gruppi *sporadici*.

$> 10^4$

pagine

~ 500

articoli

> 100

autori

~ 50

anni di lavoro

26

gruppi sporadici

- ▶ **1983** — Gorenstein annuncia il completamento della dimostrazione;
- ▶ **2004** — Aschbacher e Smith chiudono la lacuna sui gruppi *quasithin*: altre 1221 pagine, due volumi;
- ▶ **oggi** — è ancora in corso la *second-generation proof* (Gorenstein–Lyons–Solomon), pensata per renderla finalmente leggibile.

Nessuno l'ha letta tutta

💡 Il vero problema non è scriverla

Nessuno ha letto e verificato l'intera dimostrazione: la conoscenza è distribuita fra decine di specialisti, e la generazione che l'ha costruita si sta ritirando.

- ▶ la correttezza poggia su un **consenso sociale**, non su una verifica completa: è un oggetto *che nessuno possiede per intero*;
- ▶ quanti risultati della letteratura dipendono dalla classificazione senza che nessuno possa risalire alla catena completa?

⚠️ E c'è un estremo peggiore

Il problema dei *tripli pitagorici booleani* (Heule, Kullmann e Marek, 2016) l'ha risolto un SAT solver: il certificato occupa **200 terabyte**. Qui non è che nessuno l'abbia letta — è che nessuno *potrà* mai.

Chi verifica i verificatori?

Il caso della **congettura di Keplero** (Thomas Hales).

- ▶ **1998**: dimostrazione di ~300 pagine più codice, sottomessa agli *Annals of Mathematics*;
- ▶ 12 referee, **quattro anni** di lavoro; verdetto finale: «siamo **certi al 99%**», ma senza poter certificare i calcoli al calcolatore;
- ▶ **2005**: pubblicata comunque, con quella riserva a verbale.

The news from the referees is bad... They have run out of energy to devote to the problem. [Fejes Tóth] thinks that this situation will occur more and more often in mathematics. The mathematical community will have to get used to this state of affairs.

— Robert MacPherson, editor degli *Annals*

Tre reazioni possibili

LA RIVISTA

Gli *Annals* smettono di provarci: da allora non tentano più di verificare la correttezza del codice sottomesso.

Cioè: si pubblica, e la responsabilità passa altrove.

L'AUTORE

Hales avvia **Flyspeck**: una dimostrazione *formale*, completata nel **2014** con HOL Light e Isabelle.

Undici anni per trasformare il 99% in un teorema.

💡 La terza: alzare lo standard — e l'informatica l'ha già fatto

Alle conferenze A* di linguaggi di programmazione — **POPL** e sorelle — per un articolo teorica, una dimostrazione **meccanizzata** in Rocq, Lean, Agda o Isabelle è ormai lo standard di fatto: non un requisito formale, ma quasi. La spinta viene dal *POPLmark Challenge* (2005), e oggi passa dall'*artifact evaluation*.

Questo corso racconta la seconda e la terza strada. 🔗 : il caso *Kepler* è ripreso da Assia Mahboubi, *Computer-Assisted Mathematics*, TYPES 2026

Non è solo una questione di scala

Fin qui: dimostrazioni troppo grandi. Ma il punto più scomodo è un altro — **la letteratura contiene errori**, e non sappiamo quanti.

- ▶ **Vladimir Voevodsky** scoprì che un teorema che aveva pubblicato era *falso*: un controesempio di Carlos Simpson era rimasto ignorato per quindici anni. Da lì la sua conversione alla formalizzazione, e le Univalent Foundations  *L3: il seme su HoTT*;
- ▶ **Kevin Buzzard** porta l'argomento alle estreme conseguenze: la matematica moderna poggia su risultati la cui dimostrazione completa pochissimi hanno verificato. Da qui il progetto di formalizzare in Lean **l'ultimo teorema di Fermat**, iniziato nel 2024.

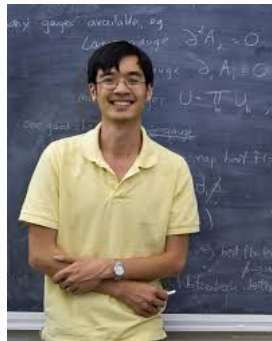
Notare la differenza rispetto a Tao: qui non c'entra l'IA. Il problema esisteva prima, e riguarda ciò che crediamo di sapere già.

Il cambio di paradigma di Terence Tao

- ▶ da osservatore scettico a utilizzatore quotidiano: Lean, GitHub Copilot e i *blueprint* collaborativi sono entrati nel suo modo di lavorare;
- ▶  L5: il progetto PFR è nato così;
- ▶ all'ICM 2026 ha dedicato la sua conferenza pubblica proprio a questo [Tao26a].

In the future, instead of typing up our proofs, we would explain them to some GPT. And the GPT will try to formalize it in Lean as you go along.

— Terence Tao, *Scientific American*, 2023



$\lambda_i > 0, \sum \lambda_i = 1$.
 $\psi \in C_c(\mathbb{R})$.
 Question: $\exists (u_k) \subset L^\infty$ bounded and that $\psi \leq u_k \leq \psi$
 $\int \psi(u_k(x)) \psi(x) dx \rightarrow \left(\int \psi(x) dx \right)^2$?

Assume next: v is a general prob. measure.
 Choose v_j discrete s.t. $v_j \rightarrow v$.
 For each j have generating sequence $(u_k^j)_k \rightarrow v$.
 Diagonal argument: $u_{k(j)}^j \rightarrow v$ if $k(j)$ is large enough.

«I think this may be my most important theorem to date.

... Better be sure it's correct...»

Peter Scholze, 5 dicembre 2020

Il Liquid Tensor Experiment

- ▶ **5 dicembre 2020** — Scholze lancia una sfida pubblica sul blog Xena: verificare il **Teorema 9.1** delle *Lectures on Analytic Geometry* (Clausen–Scholze), che lui stesso descrive come «some very convoluted thing with many moving parts»;
- ▶ **giugno 2021** — **sei mesi dopo**, la parte di cui Scholze non era sicuro è formalizzata in Lean: una dozzina di persone, coordinate da Johan Commelin;
- ▶ **14 luglio 2022** — l'intero enunciato è verificato.

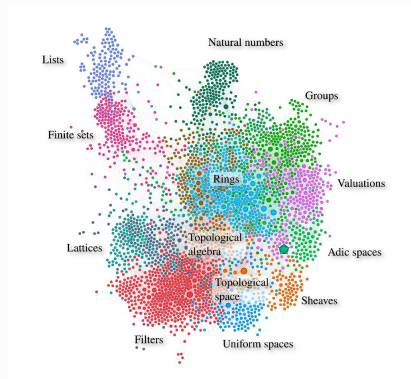


This experiment has changed drastically my impression of how capable [proof assistants] are. ... There's no real obstruction.

— Peter Scholze, giugno 2021

Mathlib: la biblioteca globale della matematica

- ▶ un deposito **unico e centralizzato** di matematica formalizzata;
- ▶ ogni nuovo teorema poggia su migliaia di lemmi già verificati: non si riparte mai da zero;
- ▶ interconnessione automatica fra settori diversi — un solo grafo di dipendenze, dagli insiemi finiti agli spazi adici.



> 200.000 teoremi · ~2 M righe · ~8000 file · >500 contributori

...ma a questa scala il problema non è più *dimostrare*: è *trovare* il lemma giusto.

🔗 L5–L6: *premise selection*

Da qui parte il corso

Come fa una macchina a *controllare* una dimostrazione?

L1

Su quale teoria si regge un proof assistant?

L2–L3

Come si dimostra, in pratica, al calcolatore?

L4–L5

E che cosa cambia adesso che c'è l'IA?

L6

Una sola idea attraversa tutto: *le dimostrazioni sono oggetti matematici*, e come tali si possono costruire, controllare e condividere.

Filo conduttore: l'isomorfismo di Curry–Howard.

Dalla logica intuizionista ai proof assistant contemporanei, attraverso una gerarchia crescente di calcoli tipati — e fino a come l'IA sta cambiando *oggi* la pratica della dimostrazione.

Struttura

Ogni lezione teorica prepara esplicitamente la successiva lezione pratica: **L1** → **L4**, **L2–L3** → **L4–L5**. Il laboratorio non è un'appendice: è il momento in cui si tocca con mano ciò che si è appena dimostrato alla lavagna.

Lezione	Blocco
L1 Logica intuizionista e isomorfismo di Curry–Howard	A
L2 Polimorfismo e il λ -cubo di Barendregt	A
L3 Martin-Löf Type Theory e il Calcolo delle Costruzioni Induttive	B
L4 Laboratorio Lean I: logica e induzione al calcolatore	C
L5 Laboratorio Lean II: strutture algebriche e Mathlib	C
L6 Proof assistant e Intelligenza Artificiale	D

Sei lezioni da due ore. Le due centrali sono di laboratorio: portate il portatile, con Lean già installato.  : *istruzioni nel repo del corso*

LEZIONE 1

Logica intuizionista e isomorfismo di Curry–Howard: il λ -calcolo semplicemente tipato

AILA 2026 — Bardonecchia

Obiettivo

Far percepire «proposizioni come tipi, dimostrazioni come programmi» non come uno slogan, ma come una corrispondenza **precisa** e **verificabile a mano**.

1. Lezione 1 — Curry–Howard

- 1.1 Che cosa vuol dire «controllare» una dimostrazione
- 1.2 Logica intuizionista in quindici minuti
- 1.3 L'isomorfismo di Curry–Howard
- 1.4 Sintassi e giudizi
- 1.5 L'esempio canonico
- 1.6 Estensioni: prodotti, somme, unità, vuoto

Blocco 1

Da Hilbert a Brouwer, da Brouwer a Curry–Howard

Abbiamo visto *perché* serve  *Introduzione: crisi della complessità, Kepler, Liquid Tensor*.
Ora la domanda tecnica: **come** è possibile?

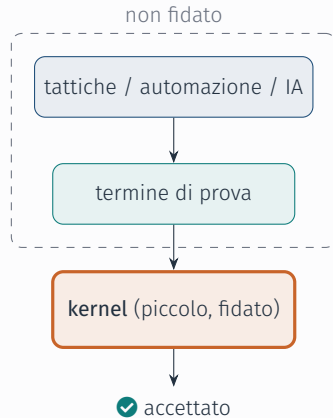
La domanda di fondo

Se una dimostrazione è un *oggetto matematico*, allora «essere una dimostrazione di A » è una proprietà *decidibile*. Chi la decide? E perché dovremmo fidarci?

Il de Bruijn criterion

Un sistema soddisfa il **de Bruijn criterion** se produce oggetti-prova controllabili da un **kernel** piccolo, indipendente e ispezionabile.

- ▶ la fiducia si concentra su poche migliaia di righe di codice;
- ▶ è possibile avere più implementazioni del kernel, e confrontarle;
- ▶ tattiche, automazione, IA *non* sono nella base fidata: qualunque cosa producano, deve passare dal kernel;
- ▶ 🔗 L6: *è esattamente ciò che rende sensato lasciar scrivere dimostrazioni a un modello di IA*



Una genealogia in 60 anni

Anno	Sistema	L'idea che ha introdotto
1967	Automath	le dimostrazioni sono termini, e una macchina li controlla
1979	LCF / ML	il tipo astratto thm : tattiche libere, correttezza garantita
1987	Edinburgh LF	un metalinguaggio unico (λII) per definire qualunque logica
1989	Coq $\rightarrow$ Rocq	CIC: Prop impredicativo, tipi induttivi, estrazione di programmi
1994	Isabelle/HOL	HOL classica, prove leggibili (Isar), automazione (Sledgehammer)
2007	Agda 2	Martin-Löf Type Theory predicativa: programmare e dimostrare sono la stessa cosa
2013	Lean	metaprogrammazione nel linguaggio stesso (Lean 4, 2021), e Mathlib

Tutti i sistemi citati soddisfano il *de Bruijn criterion*.

Tutti, tranne Isabelle/HOL, si basano sull'isomorfismo di Curry–Howard e sulla logica intuizionista.

Perché rifiutare il terzo escluso

Classicamente

$$A \vee \neg A \quad \neg\neg A \rightarrow A$$

Una dimostrazione può stabilire $\exists x. P(x)$ senza esibire alcun x .

Intuizionisticamente

$$A \vee \neg A \quad \times$$

Dimostrare $\exists x. P(x)$ significa **costruire** un testimone.

Per il pubblico di filosofia

Non è (solo) prudenza tecnica: è una tesi sul significato. Il significato di un enunciato è dato dalle sue condizioni di *dimostrazione*, non dalle sue condizioni di verità (Dummett, anti-realismo).

Brouwer–Heyting–Kolmogorov: *che cosa è* una dimostrazione?

- ▶ una dimostrazione di $A \wedge B$ è una coppia $\langle p, q \rangle$: p dimostra A , q dimostra B ;
- ▶ una dimostrazione di $A \vee B$ è una scelta di uno dei due disgiunti, più una sua dimostrazione;
- ▶ una dimostrazione di $A \rightarrow B$ è un *metodo* che trasforma dimostrazioni di A in dimostrazioni di B ;
- ▶ non esiste alcuna dimostrazione di $\perp$; e $\neg A := A \rightarrow \perp$.

💡 Intuizione

Questa è già, informalmente, **propositions as types**: la clausola BHK di ogni connettivo descrive la *struttura dei dati* dei suoi testimoni.

La tabella di corrispondenza

Logica	Tipi	Programmi
proposizione A	tipo A	specifica
dimostrazione di A	termine $t : A$	programma
$A \rightarrow B$	$A \rightarrow B$ (funzioni)	fun $x \Rightarrow \dots$
$A \wedge B$	$A \times B$ (prodotto)	coppia
$A \vee B$	$A + B$ (somma disgiunta)	unione etichettata
$\top$	1 (unità)	()
$\perp$	0 (tipo vuoto)	—
$\neg A$	$A \rightarrow \mathbf{0}$	—
normalizzazione	β -riduzione	esecuzione

Quantificatori $\forall/\exists$:  L2–L3: System F e tipi dipendenti

ALWAYS HAS BEEN

**WAIT, PROVING
IS JUST PROGRAMMING?**

Blocco 2

Il λ -calcolo semplicemente tipato

$$\begin{array}{ll} \text{Tipi} & A, B ::= \iota \mid A \rightarrow B \\ \text{Termini} & t, u ::= x \mid \lambda x:A. t \mid t u \\ \text{Contesti} & \Gamma ::= \diamond \mid \Gamma, x : A \end{array}$$

- ▶ i contesti sono liste di ipotesi con variabili *distinte*;
- ▶ il giudizio fondamentale è $\Gamma \vdash t : A$: «nel contesto Γ , il termine t ha tipo A »;
- ▶ letto logicamente: «dalle ipotesi Γ segue A , e t ne è la dimostrazione».

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A} \text{VAR}$$

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x:A. t : A \rightarrow B} \rightarrow I$$

$$\frac{\Gamma \vdash t : A \rightarrow B \quad \Gamma \vdash u : A}{\Gamma \vdash t u : B} \rightarrow E$$

Cancellando i termini...

...si ottengono *esattamente* le regole della deduzione naturale per il frammento implicativo della logica intuizionista. $\rightarrow I$ è la scarica dell'ipotesi; $\rightarrow E$ è il *modus ponens*.

$$\frac{}{(\lambda x:A. t) u \longrightarrow_{\beta} t[x := u]} \beta \qquad \frac{t \longrightarrow_{\beta} t'}{t u \longrightarrow_{\beta} t' u} \text{APP}_{\ell}$$

- ▶ **Subject reduction:** se $\Gamma \vdash t : A$ e $t \longrightarrow_{\beta} t'$ allora $\Gamma \vdash t' : A$;
- ▶ **Church–Rosser:** $\twoheadrightarrow_{\beta}$ è confluyente;
- ▶ **Strong Normalization:** ogni riduzione da un termine tipabile termina.

Le dimostrazioni degli ultimi due sono lunghe e non le faremo qui: Girard–Lafont–Taylor, capp. 4 e 6.

⚠ Consequenza logica

SN + confluenza $\Rightarrow$ forma normale unica $\Rightarrow$ coerenza: nessun t chiuso ha tipo $\perp$. *La consistenza è una proprietà computazionale.*

Subject reduction: schema della dimostrazione

Se $\Gamma \vdash t : A$ e $t \longrightarrow_{\beta} t'$, allora $\Gamma \vdash t' : A$.

IL LEMMA CHIAVE — SOSTITUZIONE

$$\Gamma, x:A \vdash u : B \text{ e } \Gamma \vdash v : A \implies \Gamma \vdash u[x := v] : B$$

per induzione sulla derivazione di $\Gamma, x:A \vdash u : B$; il caso VAR si spezza in x e $y \neq x$.

IL CASO INTERESSANTE — IL REDESSO

$\Gamma \vdash (\lambda x:A. u) v : B$	ipotesi
$\implies \Gamma \vdash \lambda x:A. u : A \rightarrow B \text{ e } \Gamma \vdash v : A$	inversione di $\rightarrow E$
$\implies \Gamma, x:A \vdash u : B$	inversione di $\rightarrow I$
$\implies \Gamma \vdash u[x := v] : B$	lemma di sostituzione

Gli altri casi sono le *congruenze*: ipotesi induttiva, e si riapplica la stessa regola.

Esempio di derivazione: Transitività dell'implicazione

Costruiamo alla lavagna il termine di tipo $(A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow (A \rightarrow C)$.

$$\frac{\frac{\frac{\frac{\frac{\Gamma \vdash f : A \rightarrow B}{\Gamma \vdash g : B \rightarrow C}}{\Gamma \vdash g (f a) : C} \rightarrow E}{f:A \rightarrow B, g:B \rightarrow C \vdash \lambda a:A. g (f a) : A \rightarrow C} \rightarrow I}{f:A \rightarrow B \vdash \lambda g. \lambda a. g (f a) : (B \rightarrow C) \rightarrow A \rightarrow C} \rightarrow I}{\vdash \lambda f. \lambda g. \lambda a. g (f a) : (A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow A \rightarrow C} \rightarrow I$$

Con $\Gamma = f:A \rightarrow B, g:B \rightarrow C, a:A$.

 L4: ritroveremo questo termine identico in Lean: **fun** a => h2 (h1 a)

Congiunzione: il prodotto $A \times B$

SINTASSI

$$A, B ::= \dots \mid A \times B \quad t, u ::= \dots \mid \langle t, u \rangle \mid \pi_1 t \mid \pi_2 t$$

TIPAGGIO

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash u : B}{\Gamma \vdash \langle t, u \rangle : A \times B} \times I$$

$$\frac{\Gamma \vdash t : A \times B}{\Gamma \vdash \pi_1 t : A} \times E_1$$

$$\frac{\Gamma \vdash t : A \times B}{\Gamma \vdash \pi_2 t : B} \times E_2$$

β -RIDUZIONE

$$\pi_1 \langle t, u \rangle \longrightarrow_{\beta} t$$

$$\pi_2 \langle t, u \rangle \longrightarrow_{\beta} u$$

Il costruttore è $\wedge I$, i distruttori sono $\wedge E$: ridurre cancella una *deviazione*.

Disgiunzione: la somma $A + B$

SINTASSI

$$A, B ::= \dots \mid A + B$$
$$t, u ::= \dots \mid \text{in}_1 t \mid \text{in}_2 t \mid \text{case } t \text{ of } [\text{in}_1 x \Rightarrow u \mid \text{in}_2 y \Rightarrow v]$$

TIPAGGIO

$$\frac{\Gamma \vdash t : A_i}{\Gamma \vdash \text{in}_i t : A_1 + A_2} +I_i \qquad \frac{\Gamma \vdash t : A + B \quad \Gamma, x:A \vdash u : C \quad \Gamma, y:B \vdash v : C}{\Gamma \vdash \text{case } t \text{ of } [\text{in}_1 x \Rightarrow u \mid \text{in}_2 y \Rightarrow v] : C} +E$$

β -RIDUZIONE

$$\text{case } (\text{in}_1 t) \text{ of } [\text{in}_1 x \Rightarrow u \mid \text{in}_2 y \Rightarrow v] \longrightarrow_{\beta} u[x := t]$$
$$\text{case } (\text{in}_2 t) \text{ of } [\text{in}_1 x \Rightarrow u \mid \text{in}_2 y \Rightarrow v] \longrightarrow_{\beta} v[y := t]$$

Il **case** è $\vee E$: per usare $A \vee B$ devo saper concludere C in *entrambi* i casi.

1 — IL VERO

$A ::= \dots \mid \mathbf{1} \quad t ::= \dots \mid \langle \rangle$

$$\frac{}{\Gamma \vdash \langle \rangle : \mathbf{1}} \mathbf{1I}$$

Un costruttore, **nessun** distruttore: niente da eliminare, nessuna β -riduzione.

0 — IL FALSO

$A ::= \dots \mid \mathbf{0} \quad t ::= \dots \mid \text{abort}_A t$

$$\frac{\Gamma \vdash t : \mathbf{0}}{\Gamma \vdash \text{abort}_A t : A} \mathbf{0E}$$

Un distruttore, **nessun** costruttore: niente da introdurre, di nuovo nessuna β -riduzione.

⚠ L'asimmetria che conta

$\mathbf{0E}$ è l'*ex falso quodlibet*, e $\neg A := A \rightarrow \mathbf{0}$. Che $\mathbf{0}$ non abbia *alcuna* regola di introduzione è ciò da cui, fra un attimo, ricaveremo la coerenza dell'intero sistema.

Le proprietà si conservano — e da qui la coerenza

Subject reduction, Church–Rosser e strong normalization valgono ancora per $\lambda_{\rightarrow}$ esteso con $\times$, $+$, **1** e **0**.

Le dimostrazioni si estendono caso per caso: ogni nuova β -regola aggiunge un caso, e nulla di più.

FORME CANONICHE

se $\vdash t : A$ t normale è...

$A \rightarrow B$ $\lambda x:A. t'$

$A \times B$ $\langle t_1, t_2 \rangle$

$A + B$ $\text{in}_1 t' \text{ o } \text{in}_2 t'$

1 $\langle \rangle$

0 **niente**

- ▶ per **SN** ogni termine chiuso ha una forma normale;
- ▶ per **CR** quella forma normale è unica;
- ▶ una forma normale *chiusa* è sempre un'introduzione del proprio tipo.

⚠ Coerenza

Nessun termine chiuso ha tipo **0**, mentre **1** ne ha esattamente uno: il sistema distingue il falso dal vero, cioè $\perp$ non è dimostrabile.

Non tutti i tipi sono abitati

Lo stesso argomento non vale solo per **0**. Dato un tipo, cercarne un abitante *normale* è una ricerca con un numero *finito* di scelte: se fallisce, il tipo è **vuoto** — cioè la proposizione non è un teorema.

ABITATI

$A \rightarrow A$ $\lambda x. x$

$A \rightarrow B \rightarrow A$ $\lambda x. \lambda y. x$

$A \rightarrow \neg\neg A$ $\lambda x. \lambda k. k x$

VUOTI

$A + \neg A$

$\neg\neg A \rightarrow A$

$((A \rightarrow B) \rightarrow A) \rightarrow A$

terzo escluso

doppia negazione

legge di Peirce

Tutti e tre a destra sono tautologie *classiche*: basta la tavola di verità.

Il confine con la logica classica

I tre schemi sono equivalenti fra loro sopra la logica intuizionista: aggiungerne uno qualunque dà esattamente la logica classica.  *L4: lo faremo in Lean — ed è coerente*

Perché la legge di Peirce è vuota

Cerchiamo un t chiuso di tipo $((A \rightarrow B) \rightarrow A) \rightarrow A$. Per SN e CR basta guardare fra i **normali**.

1. il tipo è una freccia $\Rightarrow t = \lambda k. t'$, con $k : (A \rightarrow B) \rightarrow A \vdash t' : A$;
2. A atomico $\Rightarrow t'$ non è un'introduzione: è k applicata a qualcosa, $t' = k u$ con $u : A \rightarrow B$;
3. ancora una freccia $\Rightarrow u = \lambda x:A. v$, con $k, x:A \vdash v : B$;
4. B atomico $\Rightarrow$ anche v è una variabile applicata: ma k produce A , e x non è una funzione.

Nessuna mossa.

La ricerca termina sempre: l'abitazione in $\lambda \rightarrow$ è *decidibile*, e PSPACE-completa [Sta79].

💡 E allora che cosa sarebbe un suo abitante?

Un operatore di controllo: Peirce è il tipo di **call/cc** [Gri90]. Curry–Howard arriva alla logica classica, ma il calcolo smette di essere puro.

Che cosa manca ancora?

- ▶ Abbiamo tutta la logica proposizionale intuizionista.
- ▶ **Non** abbiamo i quantificatori.
- ▶ Due strade, che percorreremo entrambe:
 - ▶ quantificare sui *tipi*: $\forall X. A$ — System F
 - ▶ quantificare sui *termini*: tipi dipendenti
- ▶ Il λ -cubo di Barendregt organizzerà queste direzioni in una mappa unica.

 L2: *polimorfismo*

 L2–L3: $\lambda P, MLTT$

Da provare prima di L4

Costruire i termini di tipo $A \rightarrow \neg\neg A$ e $\neg\neg\neg A \rightarrow \neg A$.
Poi rifare per $\neg\neg A \rightarrow A$ l'argomento della legge di Peirce.

- ▶ M. H. Sørensen e P. Urzyczyn. *Lectures on the Curry–Howard Isomorphism*. Vol. 149. Studies in Logic and the Foundations of Mathematics. Elsevier, 2006
- ▶ J.-Y. Girard, Y. Lafont e P. Taylor. *Proofs and Types*. Vol. 7. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 1989
- ▶ B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002 — capitoli 9 e 11.
- ▶ T. G. Griffin. «A Formulae-as-Types Notion of Control». In: *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '90)*. ACM, 1990, pp. 47–58 — il contenuto computazionale della logica classica.

LEZIONE 2

Polimorfismo e il λ -cubo di Barendregt: verso il Calcolo delle Costruzioni

AILA 2026 — Bardonecchia

Obiettivo

Mostrare che MLTT e CoC non sono sistemi isolati, ma **punti di una mappa sistematica a tre assi**: uno scheletro mentale per orientarsi in qualunque proof assistant si incontrerà in futuro.

2. Lezione 2 — λ -cubo

2.1 Quantificazione universale sui tipi

2.2 Espressività: le codifiche di Church

2.3 L'altra strada: quantificare sugli individui

2.4 Tre assi di generalizzazione

2.5 Il cubo delle logiche

2.6 Pure Type Systems

2.7 Il vertice: il Calcolo delle Costruzioni

2.8 Il paradosso di Girard, e come lo si evita

Blocco 1

System F ($\lambda 2$)

Dal problema all'idea

In STLC la funzione identità va scritta una volta per ogni tipo:

$$\lambda x:A. x : A \rightarrow A \quad \lambda x:B. x : B \rightarrow B \quad \dots$$

System F (Girard 1972, Reynolds 1974) internalizza il «per ogni tipo»:

$$\begin{aligned} A, B &::= X \mid A \rightarrow B \mid \forall X. A \\ t, u &::= x \mid \lambda x:A. t \mid t u \mid \Lambda X. t \mid t[A] \end{aligned}$$

$$\Lambda X. \lambda x:X. x : \forall X. X \rightarrow X$$

i Lo conoscete già: è il **polimorfismo**

In Haskell `id :: a -> a` è $\forall X. X \rightarrow X$: il **forall** resta implicito, e le applicazioni di tipo le inferisce il compilatore.

ML e Haskell: quanto System F ci sta dentro?

Il *Core* di GHC è una variante di System F: ΛX e $t[A]$ ci sono, li inserisce il compilatore.

RANGO — dove possono stare i $\forall$

1 (*prenex*): $\forall X. X \rightarrow X$

ML, e Haskell di default

2: $(\forall X. X \rightarrow X) \rightarrow \mathbb{N}$

Haskell con **RankNTypes**, ma va annotato

Sistema	Inferenza dei tipi
Hindley–Milner (rango 1)	decidibile, con <i>tipi principali</i> (Damas–Milner 1982)
System F di rango 2	ancora decidibile; dal rango 3, no
System F pieno, alla Curry	indecidibile (Wells 1994)

Il punto

Alla **Church** — con Λ e $t[A]$ scritti a mano, come faremo noi — il type checking è immediato: è la scelta dei proof assistant.

$$\frac{\Gamma, X \vdash t : A \quad X \notin \text{FV}(\Gamma)}{\Gamma \vdash \lambda X. t : \forall X. A} \forall I$$

$$\frac{\Gamma \vdash t : \forall X. A}{\Gamma \vdash t [B] : A[X := B]} \forall E$$

$$(\lambda X. t) [B] \longrightarrow_{\beta} t[X := B]$$

Lettura logica

È la logica proposizionale intuizionista del *second'ordine*: si quantifica su proposizioni. La condizione $X \notin \text{FV}(\Gamma)$ è la consueta *eigenvariable condition*.

Tutto è una funzione

I TIPI

$\text{Bool} := \forall X. X \rightarrow X \rightarrow X$

$\text{Nat} := \forall X. (X \rightarrow X) \rightarrow X \rightarrow X$

$\text{List } A := \forall X. (A \rightarrow X \rightarrow X) \rightarrow X \rightarrow X$

I COSTRUTTORI — sono i termini che *scelgono* un argomento

$\text{true} := \Lambda X. \lambda x:X. \lambda y:X. x$

$\text{zero} := \Lambda X. \lambda f:X \rightarrow X. \lambda z:X. z$

$\text{false} := \Lambda X. \lambda x:X. \lambda y:X. y$

$\text{succ} := \lambda n:\text{Nat}. \Lambda X. \lambda f. \lambda z. f (n [X] f z)$

L'ELIMINAZIONE — basta *applicare* il dato

$\text{if } b \text{ then } u \text{ else } v := b [C] u v \quad \underline{n} [C] f z \rightarrow_{\beta} f^n(z)$

Perché proprio quei tipi? Il più piccolo insieme chiuso

Al second'ordine, $\text{Nat} = \forall X. (X \rightarrow X) \rightarrow X \rightarrow X$ dice:

n appartiene a ogni insieme X che contenga lo zero e sia chiuso rispetto al successore.

I due costruttori si impacchettano in una sola mappa:

$$\underbrace{(X \rightarrow X)}_{\text{successore}} \rightarrow \underbrace{X}_{\text{zero}} \rightarrow X \quad \cong \quad (X + \mathbf{1} \rightarrow X) \rightarrow X$$
$$\text{Nat} = \bigcap \{ X : X + \mathbf{1} \rightarrow X \}$$

💡 Il più piccolo insieme chiuso rispetto ai costruttori

Un'intersezione di insiemi chiusi è chiusa ed è contenuta in tutti: è dunque **il minimo**.
È la definizione di $\mathbb{N}$ di Dedekind (1888), scritta come tipo.

In generale $T := \forall X. (F(X) \rightarrow X) \rightarrow X$, con $F(X) = X + \mathbf{1}$ per Nat : categorialmente, l'algebra iniziale di F .
🌀 L3: si ricava l'iterazione, non l'induzione

Fin dove arriva System F?

Sistema	Funzioni $\mathbb{N} \rightarrow \mathbb{N}$ rappresentabili
$\lambda_{\rightarrow}$ (STLC)	i <i>polinomi estesi</i> — pochissimo
Sistema T di Gödel	le totali dimostrabili in PA (prim'ordine)
System F	le totali dimostrabili in PA₂ (second'ordine)

Girard 1972 (GLT, cap. 15); fra queste c'è Ackermann, che non è ricorsiva primitiva.

⚠ Il paradosso apparente

La classe è enorme, eppure **ogni** termine tipabile termina. Ma la SN di System F non si dimostra *dentro* PA₂: da essa segue Con(PA₂), e per il secondo teorema di Gödel nessuna teoria coerente così forte dimostra la propria coerenza.

Con(PA₂) è l'enunciato *aritmetico* che, via numerazione di Gödel, dice «PA₂ non dimostra $0 = 1$ ». Girard aggira l'ostacolo con i *candidati di riducibilità*: impredicativi.

⚠ Il punto delicato

In $\forall X.A$ la variabile X spazia su *tutti* i tipi — incluso $\forall X.A$ stesso. La definizione è **impredicativa**: definisce un oggetto quantificando su una totalità che lo contiene.

- ▶ non è un difetto: è ciò che dà a System F la sua espressività;
- ▶ **ma** è incompatibile con universi «ingenui»: l'assunzione $\text{Type} : \text{Type}$ rende il sistema incoerente (**paradosso di Girard**);
- ▶ due risposte diverse:
 - ▶ MLTT: *gerarchia predicativa* di universi;
 - ▶ CoC/CIC: impredicatività confinata a un unico universo di *proposizioni*, Prop .

Blocco 2

Il λ -cubo

Tipi che dipendono da termini: λP

L'altra strada lasciata aperta a fine L1: quantificare non sui *tipi*, ma sugli **individui**.

SINTASSI — una sola aggiunta

$A, B ::= \dots \mid \forall x:A. B$ es. $\forall n:\mathbb{N}. \text{Vec } n \rightarrow \text{Vec } (n+1)$

$$\frac{\Gamma, x:A \vdash t : B}{\Gamma \vdash \lambda x:A. t : \forall x:A. B} \forall I \qquad \frac{\Gamma \vdash t : \forall x:A. B \quad \Gamma \vdash u : A}{\Gamma \vdash t u : B[x := u]} \forall E$$

Se x non compare in B , $\forall x:A. B$ è $A \rightarrow B$. In L3 si chiamerà $\Pi x:A. B$.

💡 Perché B possa parlare di x serve una novità

Un **predicato** è una famiglia di tipi indicizzata da termini: $\text{Vec} : \mathbb{N} \rightarrow \text{Tipo}$. **La vera aggiunta di λP è poter formare quel $\mathbb{N} \rightarrow \text{Tipo}$.**

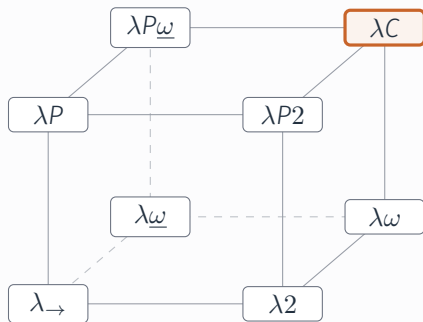
Che cosa può dipendere da che cosa

Partendo da $\lambda_{\rightarrow}$ (termini che dipendono da termini), tre direzioni indipendenti:

Dipendenza	Sistema	Che cos'è
termini da tipi	λ_2	polimorfismo (System F)
tipi da tipi	$\lambda_{\underline{\omega}}$	operatori di tipo, costruttori
tipi da termini	λ_P	tipi dipendenti

Le tre direzioni sono **ortogonali**: si possono combinare liberamente, ottenendo $2^3 = 8$ sistemi.

Il λ -cubo di Barendregt



$\rightarrow$ termini da tipi

$\uparrow$ tipi da termini

$\nearrow$ tipi da tipi

λC = Calcolo delle Costruzioni: tutte e tre le dimensioni.

Il cubo, letto come logica

Curry–Howard non si ferma alla logica proposizionale: ogni asse del cubo aggiunge un pezzo preciso di **potere logico**.

Asse	Nel calcolo	Nella logica
$\lambda 2$	termini da tipi	quantificazione sulle <i>proposizioni</i> (2° ordine)
$\lambda \omega$	tipi da tipi	<i>costruttori</i> di proposizioni (ordine superiore)
λP	tipi da termini	predicati e quantificazione sugli <i>individui</i>

Nota

Sono logiche **minimali intuizioniste**: i soli connettivi primitivi sono $\rightarrow$ e $\forall$. Gli altri o si definiscono al second'ordine $\otimes : A \vee B := \forall X. (A \rightarrow X) \rightarrow (B \rightarrow X) \rightarrow X$, o si aggiungono come costanti.

Il cubo delle logiche di Barendregt

Sistema	Logica	
$\lambda_{\rightarrow}$	PROP	proposizionale (minimale)
λ_2	PROP2	proposizionale del 2° ordine
$\lambda_{\underline{\omega}}$	PROP $\underline{\omega}$	... debolmente di ordine superiore
λ_{ω}	PROP ω	... di ordine superiore
λP	PRED	dei predicati (prim'ordine) — è la base di LF
λP_2	PRED2	dei predicati del 2° ordine
$\lambda P_{\underline{\omega}}$	PRED $\underline{\omega}$	... debolmente di ordine superiore
λC	PRED ω	dei predicati di ordine superiore (HOL)

Le immersioni sono *complete*: una formula è dimostrabile se e solo se il tipo corrispondente è abitato (Geuvers 1993).

λP **rappresenta** le altre logiche — è il nucleo di LF  L1: Harper, Honsell e Plotkin, 1987;
 λC invece le **contiene**.

Un meta-framework: i PTS

Un **Pure Type System** è una tripla $(\mathcal{S}, \mathcal{A}, \mathcal{R})$:

- ▶ $\mathcal{S}$: le *sorti* (es. $\{*, \square\}$);
- ▶ $\mathcal{A} \subseteq \mathcal{S}^2$: gli *assiomi* (es. $* : \square$);
- ▶ $\mathcal{R} \subseteq \mathcal{S}^3$: le *regole* di formazione dei prodotti.

$$\frac{\Gamma \vdash A : s_1 \quad \Gamma, x:A \vdash B : s_2 \quad (s_1, s_2, s_3) \in \mathcal{R}}{\Gamma \vdash \Pi x:A. B : s_3} \text{ PROD}$$

i Come si leggono le due sorti

$*$ è **Type**, il tipo dei tipi: $\mathbb{N} : *, \mathbb{N} \rightarrow \mathbb{N} : *$.

$\square$ è **Kind**, il tipo di $*$: l'assioma è $* : \square$; e con $(\square, \square)$ anche $* \rightarrow * : \square$.

In Rocq e Lean i nomi si spostano: l' $*$ di λC è Prop, e $\square$ è Type.

Il cubo, sistematicamente

i Nota

Otto vertici variando *soltanto* $\mathcal{R}$: $\mathcal{S} = \{*, \square\}$ e $\mathcal{A} = \{(*, \square)\}$ restano fissi.

Sistema	$(*, *)$	$(\square, *)$	$(\square, \square)$	$(*, \square)$
$\lambda_{\rightarrow}$	✓			
λ_2	✓	✓		
$\lambda_{\underline{\omega}}$	✓		✓	
λ_{ω}	✓	✓	✓	
λ_P	✓			✓
λ_{P^2}	✓	✓		✓
$\lambda_{P\underline{\omega}}$	✓		✓	✓
λ_C	✓	✓	✓	✓

(s_1, s_2) abbrevia (s_1, s_2, s_2) . $(*, *)$ è sempre presente: è $\lambda_{\rightarrow}$.

- ▶ combina polimorfismo, operatori di tipo e tipi dipendenti;
- ▶ un'unica costruzione, il **prodotto dipendente** $\prod x:A. B$, subsume $\rightarrow$, $\forall$ e la quantificazione sui tipi;
- ▶ fortemente normalizzante e coerente;
- ▶ $\text{Prop} (= *)$ è **impredicativo**: $\forall X:\text{Prop}. X$ è ancora in Prop .

💡 Perché è il punto di arrivo naturale

Un solo binder, un solo giudizio, tre generalizzazioni — ma manca ancora qualcosa: come si definiscono $\mathbb{N}$, le liste, gli alberi? 🔗 *L3: tipi induttivi $\Rightarrow$ CIC*

Il paradosso di Girard

L'impredicatività non è gratis. Se si esagera, il sistema **collassa**: si costruisce un termine chiuso di tipo $\perp$.

- ▶ **Type : Type** — la scelta della prima teoria dei tipi di Martin-Löf (1971) — è incoerente: lo mostra Girard nella tesi del 1972;
- ▶ nel linguaggio dei PTS il colpevole è il **sistema U** , con tre sorti $*$: $\square$: $\triangle$ e impredicatività a *due* livelli — e basta già la sua variante ridotta U^- ;
- ▶ è la versione type-theoretic di **Burali-Forti**: si costruisce l'«insieme di tutti gli ordinali» e lo si confronta con sé stesso.

La versione minimale: Hurkens 1995

Hurkens ha ridotto la costruzione a un termine di tipo $\perp$ in λU^- lungo poche righe: è la forma in cui il paradosso circola oggi, e con cui si collauda un kernel.

Che cos'è λU^-

Un PTS con **tre** sorti: $\mathcal{S} = \{*, \square, \triangle\}$, $\mathcal{A} = \{* : \square, \square : \triangle\}$.

Regola	Che cosa permette	Già in
$(*, *)$	funzioni fra tipi	$\lambda \rightarrow$
$(\square, *)$	polimorfismo: $\forall X:\square. A$ resta un <i>tipo</i>	$\lambda 2$
$(\square, \square)$	costruttori di tipo	$\lambda \omega$
$(\triangle, \square)$	$\forall X:\square. K$ resta un <i>kind</i> : impredicatività <i>al piano di sopra</i>	U^-
$(\triangle, *)$	... e produce anche un tipo	U

⚠ Dove sta il salto

λC , il vertice del cubo, ha **due** sorti e una sola regola impredicativa. λU^- ne aggiunge una terza e ripete l'impredicatività un livello più in alto: tanto basta a costruire $\perp$.

La costruzione di Hurkens

NOTAZIONE $\wp X := X \rightarrow *$ (i «sottoinsiemi» di X) $\perp := \forall p:*.p$ $\neg A := A \rightarrow \perp$

L'UNIVERSO PARADOSSALE — possibile solo con (Δ, Ω)

$$U := \forall X:\square. ((\wp \wp X \rightarrow X) \rightarrow \wp \wp X) : \square$$

$$\tau : \wp \wp U \rightarrow U$$

$$\tau t := \lambda X. \lambda f. \lambda p. t (\lambda x:U. p (f (x X f)))$$

$$\sigma : U \rightarrow \wp \wp U$$

$$\sigma s := s U (\lambda t. \tau t)$$

LA DIAGONALE

$$\Delta := \lambda y:U. \neg (\forall p:\wp U. \sigma y p \rightarrow p (\tau (\sigma y))) : \wp U$$

$$\Omega := \tau (\lambda p:\wp U. \forall x:U. \sigma x p \rightarrow p x) : U$$

Tre lemmi elementari su Δ e Ω danno un termine *chiuso* di tipo $\perp$ — in Coq è nella libreria standard, `Coq.Logic.Hurkens`.

Prima guardia: un solo universo impredicativo, in fondo

SISTEMA U — incoerente

$$* : \Box : \Delta$$

impredicativo *sia* a livello $\Box \rightarrow *$ *sia* a livello $\Delta \rightarrow \Box$: due piani sovrapposti, e il piano di sotto può parlare di quello di sopra.

CoC / CIC — coerente

$$\text{Prop} : \text{Type}_0 : \text{Type}_1 : \text{Type}_2 : \dots$$

un **solo** universo impredicativo, Prop , e sta in **fondo**; sopra, una gerarchia *predicativa* e senza cima. Non esiste $\text{Type} : \text{Type}$.

In termini di PTS, in CoC l'unica regola impredicativa è $(\Box, *)$: si può quantificare su *tutte* le proposizioni e restare una proposizione. Ma $\Box$ non è a sua volta abitante di una sorte impredicativa — ed è precisamente il secondo piano che manca al paradosso per chiudersi.

Seconda guardia: da Prop non si estraggono dati

Non basta confinare l'impredicatività: serve anche impedire che una *dimostrazione* diventi un *dato*.

- ▶ in CIC l'eliminazione da Prop verso Type è **vietata** (salvo casi singoletto): da $h : P$ con $P : \text{Prop}$ non si costruisce un valore in Type;
- ▶ senza questa restrizione l'impredicatività riapre il paradosso — Coquand, 1986;
- ▶ il contraltare: Prop è *proof-irrelevant*, e le dimostrazioni si possono cancellare in fase di estrazione.

⚠ Non è un timore teorico

In Coq **Set** era impredicativo: con certi assiomi classici diventava contraddittorio. Dalla versione 8.0 (2004) è predicativo di default, e l'impredicatività è un'opzione da chiedere esplicitamente. 🔗 L3: Lean 4 fa la stessa scelta, con *proof irrelevance* definizionale

- ▶ H. Barendregt. «**Lambda Calculi with Types**». In: *Handbook of Logic in Computer Science*. A cura di S. Abramsky, D. M. Gabbay e T. S. E. Maibaum. Vol. 2. Oxford University Press, 1992, pp. 117–309
- ▶ T. Coquand e G. Huet. «**The Calculus of Constructions**». In: *Information and Computation* 76.2–3 (1988), pp. 95–120
- ▶ J.-Y. Girard. «**Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur**». Thèse d'État. Université Paris VII, 1972 — per System F e la normalizzazione.
- ▶ A. J. C. Hurkens. «**A Simplification of Girard's Paradox**». In: *Typed Lambda Calculi and Applications (TLCA '95)*. Vol. 902. LNCS. Springer, 1995, pp. 266–278

- ▶ T. Coquand. «**An Analysis of Girard's Paradox**». In: *Symposium on Logic in Computer Science (LICS '86)*. IEEE, 1986, pp. 227–236
- ▶ H. Geuvers. «**Logics and Type Systems**». Tesi di dott. Katholieke Universiteit Nijmegen, 1993 — per il cubo delle logiche.

LEZIONE 3

Martin-Löf Type Theory e il Calcolo delle Costruzioni Induttive

AILA 2026 — Bardonecchia

Obiettivo

Arrivare al nucleo teorico **esatto** su cui sono costruiti Lean 4 e Rocq, così che il laboratorio delle lezioni successive non sia una scatola nera.

3. Lezione 3 — MLTT e CIC

3.1 Da dove viene MLTT

3.2 L'apparato giudizionale

3.3 Tipi Π e Σ

3.4 Uguaglianza proposizionale

3.5 Universi

3.6 Tipi induttivi ed eliminatori

3.7 Che cosa si sa di MLTT

3.8 Che aspetto ha un modello

Blocco 1

Martin-Löf Type Theory

Dopo il cubo: cambiare programma

Nel cubo si parte da $\lambda \rightarrow$ e si aggiungono assi. MLTT non è un altro vertice: è un **programma diverso**.

- ▶ l'obiettivo di Martin-Löf non è un calcolo più espressivo, ma una **fondazione costruttiva della matematica**, alternativa alla teoria degli insiemi;
- ▶ nella pratica usuale ci sono *due* livelli — una logica del prim'ordine e, sopra, un universo di insiemi. Qui il livello è **uno solo**;
- ▶ una proposizione **è** un tipo: non per un isomorfismo  L1: *Curry-Howard*, ma per definizione.

💡 Un linguaggio, tre mestieri

Lo stesso formalismo è insieme *logica*, *linguaggio di programmazione* e *fondazione*. E le regole non si postulano: si giustificano spiegando *che cosa significa* sapere che a è un elemento di A — la BHK di L1, promossa a semantica ufficiale.

- ▶ 1971 — prima versione, con $\text{Type} : \text{Type}$. **Incoerente**: è il bersaglio del paradosso di Girard  L2: *ce ne siamo appena occupati*;
- ▶ 1972–73 — versione **predicativa**: entra la gerarchia di universi. È la MLTT di cui parleremo;
- ▶ 1980 — lezioni a Padova, raccolte da Giovanni Sambin; diventano *Intuitionistic Type Theory*, Bibliopolis 1984.

Due varianti, ancora oggi

Estensionale: l'uguaglianza proposizionale si riflette in quella giudizionale — comodo, ma il type checking diventa indecidibile (NuPRL). **Intensionale**: le due restano distinte, il controllo resta decidibile. È la strada di Agda, Idris, Rocq e Lean.

Che cosa cambia rispetto a λC

	λC , il vertice del cubo	MLTT
si parte da	un λ -calcolo da estendere	i giudizi
le proposizioni	sono tipi, per Curry–Howard	<i>sono</i> tipi, per definizione
l'uguaglianza	un giudizio	anche un tipo
gli universi	Prop impredicativo	gerarchia predicativa
i dati	si codificano  L2: $\forall X. \dots$	tipi induttivi primitivi

Le ultime tre righe sono il programma del blocco. E per arrivarci si riparte da capo: non dai termini, ma da **che cosa si è autorizzati ad asserire**.

Quattro forme di giudizio

$\Gamma \text{ ctx}$	Γ è un contesto ben formato
$\Gamma \vdash A \text{ type}$	A è un tipo
$\Gamma \vdash a : A$	a è un elemento di A
$\Gamma \vdash a \equiv b : A$	a e b sono defnizionalmente uguali

Il punto da non perdere

L'uguaglianza $\equiv$ è un **giudizio**: è decidibile, la controlla il kernel, non si dimostra. Presto incontreremo un'uguaglianza che è invece una **proposizione**, cioè un tipo abitabile o meno.

Come si costruisce un contesto

FORMAZIONE DEI CONTESTI

$$\frac{}{\diamond \text{ ctx}} \text{CTX-}\diamond \qquad \frac{\Gamma \text{ ctx} \quad \Gamma \vdash A \text{ type} \quad x \notin \text{dom}(\Gamma)}{\Gamma, x:A \text{ ctx}} \text{CTX-EXT}$$

ED È QUI CHE SERVE — LA REGOLA DELLA VARIABILE

$$\frac{\Gamma, x:A, \Delta \text{ ctx}}{\Gamma, x:A, \Delta \vdash x : A} \text{VAR}$$

Un contesto non è una lista di ipotesi

La premessa di CTX-EXT è $\Gamma \vdash A \text{ type}$ *nel contesto* Γ : ogni tipo può dipendere dalle variabili precedenti. Un contesto è un **telescopio**: $n:\mathbb{N}$, $v:\text{Vec } \mathbb{N} \ n$, $p: n > 0$.

Lo schema a quattro tempi

Ogni costruttore di tipi si presenta con quattro gruppi di regole:

1. **formazione**: quando il tipo è ben formato;
2. **introduzione**: come si costruiscono i suoi elementi;
3. **eliminazione**: come si usano i suoi elementi;
4. **computazione** (β): che cosa succede quando l'eliminazione incontra l'introduzione.

Nota

È la stessa disciplina della deduzione naturale (*harmony* fra intro ed elim). Riconoscerla rende meccanico leggere le regole di qualunque tipo, incluso quelli che definiremo noi in Lean.  L4: **inductive**, **cases**, **induction**

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma, x:A \vdash B \text{ type}}{\Gamma \vdash \Pi x:A. B \text{ type}} \Pi\text{-F}$$

$$\frac{\Gamma, x:A \vdash b : B}{\Gamma \vdash \lambda x. b : \Pi x:A. B} \Pi\text{-I}$$

$$\frac{\Gamma \vdash f : \Pi x:A. B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[x := a]} \Pi\text{-E}$$

$$\frac{\Gamma, x:A \vdash b : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda x. b) a \equiv b[x := a] : B[x := a]} \Pi\text{-}\beta$$

- ▶ se $x \notin \text{FV}(B)$ si riottiene $A \rightarrow B$;
- ▶ lettura logica: $\Pi x:A. B(x)$ è $\forall x:A. B(x)$.

Σ : coppie dipendenti = $\exists$

SINTASSI

$$A, B ::= \dots \mid \Sigma x:A. B \quad t ::= \dots \mid \langle a, b \rangle \mid \text{ind}_{\Sigma}(p, c)$$

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma, x:A \vdash B \text{ type}}{\Gamma \vdash \Sigma x:A. B \text{ type}} \Sigma\text{-F}$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[x := a]}{\Gamma \vdash \langle a, b \rangle : \Sigma x:A. B} \Sigma\text{-I}$$

- ▶ in $\Sigma\text{-I}$ il secondo componente ha tipo $B[x := a]$: **dipende dal primo**. È tutta qui la differenza col prodotto;
- ▶ se $x \notin \text{FV}(B)$ si riottiene $A \times B$: la congiunzione di L1 è il caso non dipendente.

$$\frac{\Gamma \vdash p : \Sigma x:A. B \quad \Gamma, x:A, y:B \vdash c : C(\langle x, y \rangle)}{\Gamma \vdash \text{ind}_{\Sigma}(p, c) : C(p)} \quad \Sigma\text{-E}$$

$$\text{ind}_{\Sigma}(\langle a, b \rangle, c) \equiv c[x := a][y := b]$$

- ▶ per *usare* una coppia basta saper trattare il caso $\langle x, y \rangle$: è l'unico modo di costruirla
🔗 L1: esattamente come il *case* per $+$;
- ▶ C è il **motive**: una *famiglia* indicizzata da $p : \Sigma x:A. B$, non un tipo fisso. Con C costante si riottiene l'eliminazione ordinaria, cioè l'*uncurry*.

Nota

Il **motive** tornerà nel blocco 2, in ogni ricorsore generato dal kernel.

Le proiezioni, e un $\exists$ che consegna il testimone

Le proiezioni non sono primitive: si **derivano** dall'eliminatore.

$$\begin{array}{ll}\pi_1 p := \text{ind}_\Sigma(p, \lambda x. \lambda y. x) & : A \\ \pi_2 p := \text{ind}_\Sigma(p, \lambda x. \lambda y. y) & : B[x := \pi_1 p]\end{array}$$

Il tipo di π_2 menziona π_1 : per costruirlo serve l'eliminazione *dipendente*, con $C(p) := B[x := \pi_1 p]$.
Un eliminatore non dipendente darebbe solo π_1 .

$\Sigma x:A. B(x)$ è $\exists x:A. B(x)$, ma un $\exists$ **costruttivo**: π_1 estrae *sempre* il testimone.

💡 Assioma della scelta

In MLTT $\prod x:A. \Sigma y:B. R(x, y) \rightarrow \Sigma f:A \rightarrow B. \prod x:A. R(x, f x)$ è *dimostrabile*.
Perché non è «l'» assioma della scelta?

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash \text{Id}_A(a, b) \text{ type}} \text{Id-F}$$

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{refl}_a : \text{Id}_A(a, a)} \text{Id-I}$$

ELIMINAZIONE — J, L'INDUZIONE SUI CAMMINI

$$\frac{\Gamma \vdash p : \text{Id}_A(a, b) \quad \Gamma, z:A \vdash c : C(z, z, \text{refl}_z)}{\Gamma \vdash J(c, p) : C(a, b, p)} \text{Id-E}$$

$$J(c, \text{refl}_a) \equiv c[z := a]$$

Il **motive** $C(x, y, p)$ dipende dai due estremi *e dal cammino*. E la premessa chiede c solo nel caso refl : **è l'unico costruttore, e basta saper trattare quello.**

Due uguaglianze, e non sono la stessa

	$a \equiv b : A$	$\text{Id}_A(a, b)$
che cos'è	giudizio	tipo (proposizione)
chi decide	il kernel, calcolando	si deve <i>dimostrare</i>
come si usa	silenziosamente, ovunque	con J
esempio	$2 + 2 \equiv 4$	$\forall n. 0 + n = n$

⚠ Dove le due nozioni divergono

Su termini *chiusi* coincidono. Con variabili libere no: $n + 0 \equiv n$ vale per definizione, ma $0 + n = n$ va dimostrato per induzione — dipende da quale argomento consuma la ricorsione.

🔗 L4: **rfl** contro **induction**: è esattamente questa differenza

$$\text{Type}_0 : \text{Type}_1 : \text{Type}_2 : \dots$$

- ▶ ogni tipo abita un universo; nessun universo abita se stesso;
- ▶ **cumulatività**: $A : \text{Type}_i$ implica $A : \text{Type}_j$ per $j \geq i$. Rocq ce l'ha, **Lean 4 no**;
- ▶ $\prod x:A. B$ con $A : \text{Type}_i, B : \text{Type}_j$ vive in $\text{Type}_{\max(i,j)}$ — **predicativo**.

In Lean il servizio della cumulatività lo rende il **polimorfismo di universo**: $\text{Type } u$ con u variabile di livello — una definizione sola, valida a ogni livello.

⚠ Perché serve

Con $\text{Type} : \text{Type}$ si riproduce Russell/Burali-Forti in veste type-theoretic: il **paradosso di Girard** [🔗](#) *L2: ci siamo appena stati*. La gerarchia è il prezzo della coerenza.

$$\frac{}{\vdash \text{zero} : \mathbb{N}} \qquad \frac{\vdash n : \mathbb{N}}{\vdash \text{succ } n : \mathbb{N}}$$

L'eliminatore (*ricorsore*) è il principio di induzione:

$$\text{ind}_{\mathbb{N}} : \Pi C : \mathbb{N} \rightarrow \mathcal{U}. C(\text{zero}) \rightarrow (\Pi n : \mathbb{N}. C(n) \rightarrow C(\text{succ } n)) \rightarrow \Pi n : \mathbb{N}. C(n)$$

$$\text{ind}_{\mathbb{N}}(C, c_0, c_s, \text{zero}) \equiv c_0 \qquad \text{ind}_{\mathbb{N}}(C, c_0, c_s, \text{succ } n) \equiv c_s \, n \, (\text{ind}_{\mathbb{N}}(C, c_0, c_s, n))$$

Nota

Scriverlo esplicitamente alla lavagna: in L4 questo diventa letteralmente **induction** `n with | zero => ... | succ n ih => ....` La `ih` è l'argomento $C(n)$ di c_s .

Bool

true, false : Bool

List A

nil, cons : $A \rightarrow \text{List } A \rightarrow \text{List } A$

Vec A n

(famiglia *indicizzata*)

vnil : Vec A 0

vcons : $A \rightarrow \text{Vec } A\ n \rightarrow \text{Vec } A\ (n+1)$

L'indice *dipende da un termine*: qui i tipi dipendenti guadagnano il loro stipendio.

Esercizio

Scrivere il tipo dell'eliminatore di List A. Confrontarlo con l'enunciato dell'induzione strutturale sulle liste.

- ▶ **normalizzazione** $\Rightarrow \equiv$ decidibile $\Rightarrow$ **type checking decidibile** — per la teoria *intensionale*; nell'estensionale no;
- ▶ **canonicità**: ogni termine chiuso di tipo $\mathbb{N}$ si riduce a un numerale — da qui *disgiunzione ed esistenza*;
- ▶ **Aczel [Acz78]**: la teoria degli insiemi costruttiva **CZF** si interpreta dentro MLTT, con un universo e i **W-tipi** — gli alberi ben fondati $\forall x:A. B(x)$;
- ▶ **Hofmann [Hof95]**: la teoria estensionale è *conservativa* su quella intensionale + K + **funext**.

CZF non è ZFC: intuizionista e *predicativa*, senza insieme potenza. Ma $\text{CZF} + \text{terzo escluso} = \text{ZF}$.

💡 Che cosa dicono, in due frasi

Aczel: MLTT non *assomiglia* a una teoria degli insiemi, ne *contiene* una.

Hofmann: l'estensionalità non compra potere, compra brevità.

Blocco 2

Il Calcolo delle Costruzioni Induttive

- ▶ **Calculus of Constructions**: prodotti dipendenti, Prop impredicativo, gerarchia Type_i predicativa  L2: il vertice λC ;
- ▶ + **scemi induttivi** alla Martin-Löf, con generazione automatica dei ricorsori (Paulin-Mohring);
- ▶ = il kernel di **Rocq** e, con alcune varianti, il type checker di **Lean 4**.

Varianti in Lean 4

Prop con *definitional proof irrelevance*; quozienti e **Classical.choice** come assiomi (Lean è coerente ma *non* puramente costruttivo di default); **Type** u con polimorfismo di universo esplicito.

Che cos'è uno schema induttivo

Un tipo induttivo si dichiara dando i **costruttori**; il kernel ne ricava tutto il resto.

```
inductive Vec ( $\alpha$  : Type u) : Nat  $\rightarrow$  Type u where    --  $\alpha$  parametro, Nat indice  
  | nil   : Vec  $\alpha$  0  
  | cons :  $\alpha \rightarrow$  Vec  $\alpha$  n  $\rightarrow$  Vec  $\alpha$  (n+1)
```

Parametri: fissi in tutti i costruttori. **Indici**: variano, e finiscono nel *motive* dell'eliminatore.

La u è una variabile di **livello**: `Vec` è dichiarato una volta e vive in ogni universo  : *polimorfismo di universo*.

Stretta positività

Il tipo che si sta definendo può comparire negli argomenti dei costruttori solo in posizione **positiva**: mai a sinistra di una freccia. Senza questo vincolo il sistema crolla.

Perché serve la positività

```
inductive Bad where
  | mk : (Bad → False) → Bad      -- Bad a SINISTRA di →: rifiutato

def f : Bad → False
  | .mk g => g (.mk g)

example : False := f (.mk f)      -- ⊥, e la riduzione non termina
```

Lean rifiuta la dichiarazione: *non-positive occurrence of the datatypes being declared.*

💡 Intuizione

Un'occorrenza negativa permette l'auto-applicazione: è lo stesso meccanismo del λ -calcolo non tipato, dove $\Omega = (\lambda x. xx)(\lambda x. xx)$ non termina. La positività è ciò che garantisce che l'induttivo sia davvero *il più piccolo* punto fisso  L2: *il minimo insieme chiuso*.

Il ricorsore, generato dal kernel

```
Nat.rec : {motive : Nat → Sort u} →  
  motive Nat.zero →  
  ((n : Nat) → motive n → motive n.succ) →  
  (t : Nat) → motive t
```

- ▶ il **motive** è una famiglia $T \rightarrow \text{Sort } u$, non un tipo fisso: è questo a rendere l'eliminazione **dipendente**;
- ▶ motive costante $\Rightarrow$ si riottiene l'iteratore  L2: la codifica impredicativa si fermava qui;
- ▶ motive in Prop: è il *principio di induzione*; in Type: è la ricorsione che costruisce dati.

Regola di computazione (ι): $\text{Nat.rec } z \ s \ (\text{succ } n) \equiv s \ n \ (\text{Nat.rec } z \ s \ n)$. In Lean **induction, cases** e **match** sono zucchero sopra **Nat.rec**.  L4: è letteralmente la stessa cosa

Anche l'uguaglianza è un tipo induttivo

```
inductive Eq {α : Sort u} (a : α) : α → Prop where -- a e' parametro, il
    secondo indice
| refl : Eq a a
```

Un solo costruttore. Il ricorsore che ne esce è la regola J del blocco 1:

```
Eq.rec : {motive : (b : α) → a = b → Sort v} →
    motive a rfl → {b : α} → (h : a = b) → motive b h
```

- **rfl** è l'unico modo di *costruire* un'uguaglianza; **rw**, **subst** e l'operatore ► sono tutti usi di **Eq.rec**;
- il tipo identità non è un'aggiunta esotica: è un induttivo come $\mathbb{N}$ o **List**.

E poiché il costruttore è uno solo, nulla obbliga $\text{Id}_A(a, b)$ ad avere *al più* un abitante: è la porta da cui entra HoTT, fra due slide.

Prop contro Type: che cosa si può eliminare

In CIC un induttivo in Prop **non** può essere eliminato verso Type: da una prova non si estraggono dati.

Tipo	Elim. verso Type	Perché
False, Eq, And	permessa	al più un costruttore, nessun dato
Or, Exists	vietata	sceglierne uno rivelerebbe informazione

i Le due facce della restrizione

Senza di essa, impredicatività + eliminazione forte = incoerenza [L2: Coquand 1986](#).
Con essa si guadagna l'**estrazione**: le prove si cancellano, e resta il programma. È anche il motivo del **noncomputable** di Lean. [L4: *Classical.choice*](#)

MLTT vs CoC vs CIC

	MLTT	CoC	CIC
prodotti dipendenti II	✓	✓	✓
universo impredicativo	✗	✓ (Prop)	✓ (Prop)
gerarchia predicativa	✓	✓	✓
tipi induttivi primitivi	✓	✗ (codifiche)	✓
proof irrelevance	✗	(opzionale)	✓ (Lean)
implementazioni	Agda, Idris	—	Rocq, Lean 4

In CoC i tipi induttivi si *codificano* (impredicativamente, alla Church) ma senza le regole η /dipendenti che servono per l'induzione: di qui la necessità di primitivizzarli.

	Rocq	Lean 4
Prop	impredicativo; proof irrelevance <i>proposizionale</i> (o SProp)	impredicativo; proof irrelevance definizionale
universi	cumulativi; polimorfismo opt-in	non cumulativi; Sort u ovunque
quozienti	setoidi, o assiomi	primitivi ($\Rightarrow$ funext è teorema)
assiomi	nessuno di serie	propext , Classical.choice , Quot.sound

💡 Il de Bruijn criterion, in pratica

Entrambi hanno un kernel piccolo e indipendente dal resto 🌀 *L1: tattiche e IA non sono nella base fidata*, e per entrambi esistono *type-checker esterni* scritti da terzi: la fiducia si può replicare.

La metateoria: perché il kernel può esistere

$SN \implies \equiv \text{decidibile} \implies \text{type checking decidibile} \implies \text{un kernel}$

- ▶ vale per la teoria **intensionale**: in quella estensionale la riflessione dell'uguaglianza rende il controllo indecidibile;
- ▶ la SN di CIC non è dimostrabile *dentro* CIC  L2: *ancora Gödel*; la dimostrazione è nella tesi di Werner, 1994.

CANONICITÀ

Ogni termine chiuso di tipo $\mathbb{N}$ si riduce a un numerale. Di qui la **proprietà della disgiunzione** e quella **dell'esistenza**: da $\vdash \exists x. P(x)$ si estrae un testimone.

Quanto è forte

Quanto a coerenza, CIC con la gerarchia di universi sta a ZFC con una successione di inaccessibili.  : *fra due slide: che aspetto ha quel modello, e perché esiste*

Il polimorfismo non è insiemistico

Che cosa è un tipo? La risposta ingenua: un insieme, e $A \rightarrow B$ l'insieme di **tutte** le funzioni da A a B . Ma questo porta a un assurdo: Reynolds [Rey84] dimostra che non esiste un modello ingenuo di System F, e quindi nemmeno di λC .

SCHEMA DELLA DIMOSTRAZIONE — Reynolds [Rey84]

- ▶ in L2: $\forall X. (FX \rightarrow X) \rightarrow X$ è l'**algebra iniziale** di F , per ogni F definibile;
- ▶ prendi $F(X) = (X \rightarrow 2) \rightarrow 2$, cioè $\wp\wp$;
- ▶ un'algebra iniziale è un punto fisso (Lambek): $A \cong \wp(\wp A)$;
- ▶ ma $|A| < |\wp A| < |\wp\wp A|$ per Cantor. **Assurdo**.

λC lo eredita perché contiene System F dentro Prop.

💡 Che cosa si fa allora

Si tengono gli insiemi e si restringe la *freccia*: PER su $\mathbb{N}$, insiemi modesti, domini. In $A \rightarrow B$ stanno solo le funzioni *realizzabili*, e $\forall$ è un'intersezione: resta piccola.

E allora perché Lean un modello ce l'ha?

Eppure Carneiro [Car19] ne costruisce uno, in $\text{ZFC} + \omega$ **inaccessibili**. Come?

PRIMA MOSSA — L'IMPREDICATIVO È Prop, E Prop È PROOF-IRRELEVANT

Una proposizione denota **al più un elemento**: $\emptyset$ se falsa, $\{\emptyset\}$ se vera. Quindi $\forall X:\text{Prop}. X$ è un'intersezione di sottosingoletti: **nessuna cardinalità esplode**.

SECONDA MOSSA — Type È PREDICATIVO

$\text{Type}_u := V_{\kappa_u}$ con κ_u **inaccessibile**: i Π non escono dall'universo, perché un **inaccessibile** è chiuso per prodotti di taglia minore.

Intuizione

La «seconda guardia» di L2 — niente eliminazione da Prop verso Type — è *esattamente* ciò che rende lecito il collasso: se non puoi osservare *quale* prova avevi, il modello le identifica tutte.

Che cosa MLTT *non* dimostra

L'ASSIOMA K — unicità delle prove di identità

$$K : \prod a:A. \prod p:\text{Id}_A(a, a). \quad p = \text{refl}_a$$

- ▶ **non** è derivabile: Hofmann e Streicher (1998) costruiscono il **modello dei gruppoidi**, dove un tipo è un gruppoide e Id l'insieme degli isomorfismi;
- ▶ lì $\text{Id}_A(a, a)$ può avere *più* abitanti: K è falso nel modello, dunque non è un teorema.

E L'ESTENSIONALITÀ FUNZIONALE

Nemmeno **funext** è derivabile. In Rocq è un assioma; in Lean è un *teorema*, perché i quozienti sono primitivi e lo si ricava da **Quot.sound**.

Non sono lacune: Id ha una struttura più ricca di quanto sembri

Un seme per il futuro: Homotopic Type Theory (HoTT)

- ▶ che cosa è $\text{Id}_A(a, b)$? In MLTT non è necessariamente un tipo con al più un abitante;
- ▶ interpretazione omotopica: A è uno *spazio*, a, b punti, $\text{Id}_A(a, b)$ lo spazio dei *cammini* da a a b , e $\text{Id}_{\text{Id}_A(a, b)}(p, q)$ le omotopie fra cammini...
- ▶ **univalenza** (Voevodsky): $(A \simeq B) \simeq \text{Id}_{\mathcal{U}}(A, B)$ — «tipi equivalenti sono uguali».

💡 Intuizione

Non lo useremo. Ma è lo sviluppo naturale di ciò che abbiamo appena visto, ed è il motivo per cui vale la pena distinguere con cura $\equiv$ da Id .

- ▶ P. Martin-Löf. *Intuitionistic Type Theory*. Studies in Proof Theory. Note di Giovanni Sambin delle lezioni di Padova, 1980. Napoli: Bibliopolis, 1984
- ▶ B. Nordström, K. Petersson e J. M. Smith. *Programming in Martin-Löf's Type Theory: An Introduction*. International Series of Monographs on Computer Science. Oxford University Press, 1990
- ▶ C. Paulin-Mohring. «Inductive Definitions in the System Coq: Rules and Properties». In: *Typed Lambda Calculi and Applications (TLCA '93)*. Vol. 664. LNCS. Springer, 1993, pp. 328–345
- ▶ The Lean Developers. *The Lean Language Reference*. 2025. URL: <https://lean-lang.org/doc/reference/latest/> — sezione sul kernel.

- ▶ P. Aczel. «**The Type Theoretic Interpretation of Constructive Set Theory**». In: *Logic Colloquium '77*. A cura di A. Macintyre, L. Pacholski e J. Paris. North-Holland, 1978, pp. 55–66
- ▶ M. Hofmann. «**Extensional Concepts in Intensional Type Theory**». Pubblicata da Springer, Distinguished Dissertations, 1997. Tesi di dott. University of Edinburgh, 1995
- ▶ B. Werner. «**Une Théorie des Constructions Inductives**». Tesi di dott. Université Paris VII, 1994 — la metateoria di CIC.
- ▶ J. C. Reynolds. «**Polymorphism is not Set-Theoretic**». In: *Semantics of Data Types*. Vol. 173. LNCS. Springer, 1984, pp. 145–156

- ▶ B. Werner. «Sets in Types, Types in Sets». In: *Theoretical Aspects of Computer Software (TACS '97)*. Vol. 1281. LNCS. Springer, 1997, pp. 530–546 — il modello insiemistico di CIC.
- ▶ M. Carneiro. «The Type Theory of Lean». Tesi di laurea mag. Carnegie Mellon University, 2019 — e quello di Lean.
- ▶ M. Hofmann e T. Streicher. «The Groupoid Interpretation of Type Theory». In: *Twenty-Five Years of Constructive Type Theory*. Vol. 36. Oxford Logic Guides. Oxford University Press, 1998, pp. 83–111
- ▶ The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Princeton: Institute for Advanced Study, 2013. URL: <https://homotopytypetheory.org/book/> — per chi vuole seguire il seme.

LEZIONE 4

Laboratorio Lean I: logica e induzione al calcolatore

AILA 2026 — Bardonecchia

Obiettivo

Far **toccare con mano** esattamente ciò che è stato dimostrato astrattamente in L1 e L3.

Su che cosa poggia Lean

Il nucleo è **CIC**  L3: Prop *impredicativo*, *induttivi*, *universi*, con **proof irrelevance** **definizionale**. Sopra, *tre* assiomi — e sono tutti.

I TRE ASSIOMI — `#print axioms` li elenca

`propext` $(a \leftrightarrow b) \rightarrow a = b$

`Quot.sound` $\forall r : \alpha \rightarrow \alpha \rightarrow \text{Prop}, \forall a b : \alpha, r a b \rightarrow [a]_r = [b]_r$

`Classical.choice` $\text{Nonempty } \alpha \rightarrow \alpha$

Che cosa *non* è un assioma

La proof irrelevance non sta in questa lista: non è un assioma, è una **regola del kernel**. Due prove della stessa proposizione sono uguali *per definizione*, senza che nessuno lo debba assumere.

Che cosa ne segue: siamo in HOL classica

Da quei tre assiomi, più la proof irrelevance, discendono **come teoremi** tutti gli assunti della matematica classica.

Quot.sound	⇒	funext
proof irrelevance	⇒	UIP / K : $\forall h_1 h_2 : a = b, h_1 = h_2$, per rfl
scelta + propext + funext	⇒	Classical.em  : Diaconescu

In MLTT K non è derivabile  L3: Hofmann–Streicher; qui è gratis. Per questo Lean e HoTT non possono stare insieme.

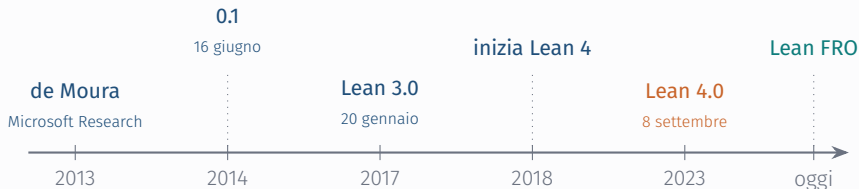
Dove ci troviamo, in pratica

Prop è booleana a due valori, le funzioni sono estensionali, c'è la scelta: sono *esattamente* gli assiomi della **logica classica di ordine superiore**. Sotto restano però tipi dipendenti, universi e induttivi — e `#print axioms` tiene la distinzione **ispezionabile**, cosa che in HOL non si può fare.

to combine the high level of trust provided by a small, independently-implementable logical kernel with the convenience and automation of tools like SMT solvers

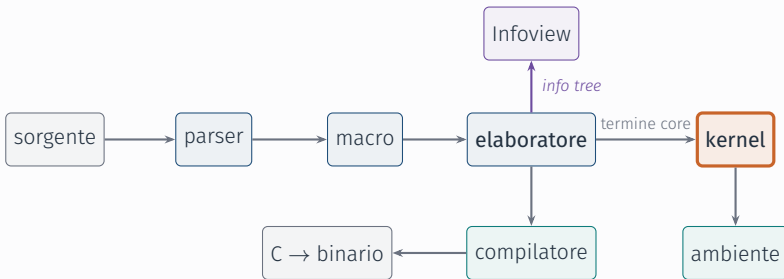
— Lean Language Reference

- ▶ lo stesso linguaggio in cui si dimostrano teoremi è quello in cui si scrivono programmi — e in cui è scritto Lean stesso: circa il **90%** dell'implementazione è in Lean;
- ▶ tattiche, notazioni e nuovi comandi si definiscono *dentro* il linguaggio  L1: la linea LCF, senza metalinguaggio separato;
- ▶ sotto, il nucleo è CIC  L3: Prop, induttivi, quozienti.



- ▶ **Lean 3** porta l'adozione di massa fra i matematici: Mathlib supera il milione di righe;
- ▶ **Lean 4** è riscritto in Lean. Nonostante Mathlib sia cresciuta del 50% durante il porting, la controlla *più in fretta* di quanto Lean 3 controllasse la versione più piccola;
- ▶ dal 2023 lo sviluppo è alla **Lean FRO**, una non-profit fondata da de Moura e Ullrich.

Che cosa succede quando compili un .lean



⚠ Dove sta la fiducia

L'elaboratore è enorme — unificazione, type class, inferenza, tattiche — e **non** è nella base fidata. Il kernel riceve un termine già esplicito: non unifica e non ha un controllo di terminazione, perché la ricorsione è già stata tradotta in usi dei ricorsori [L3](#): *Nat.rec*.

Lean non compila il file *tutto insieme*: legge un comando, lo elabora, lo fa controllare al kernel, aggiorna l'ambiente, e passa al successivo.

- ▶ per questo una **notation** o una **macro** è utilizzabile *dalla riga dopo*: il parser stesso è stato esteso;
- ▶ per questo l'editor può dirti che cosa sa *in quel punto* del file: gli *info tree* prodotti dall'elaboratore sono ciò che alimenta l'Infoview;
- ▶ per questo un errore a metà file non impedisce di lavorare sul resto.

La compilazione vera e propria è un ramo a parte: un file C per modulo, poi codice nativo. Serve a *eseguire*, non a *fidarsi*.

Notazione: come si scrivono i simboli

Si digita la **barra rovesciata** seguita dal nome, poi spazio o tab. Passando il mouse su un simbolo, VS Code mostra l'abbreviazione che lo produce.

$\rightarrow$	<code>\to \r</code>	$\forall$	<code>\forall</code>	<code>\forall</code>	<code>\forall</code>	$\langle \rangle$	<code>\< \></code>
λ	<code>\lambda \l</code>	$\exists$	<code>\exists</code>	<code>\exists</code>	<code>\exists</code>	$\cdot$	<code>\cdot</code>
$\wedge$	<code>\and</code>	$\neg$	<code>\not</code>	<code>\not</code>	<code>\not</code>	$^{-1}$	<code>\inv</code>
$\vee$	<code>\or</code>	$\neq$	<code>\ne</code>	<code>\ne</code>	<code>\ne</code>	$\mathbb{N} \mathbb{R}$	<code>\N \R</code>
$\leftrightarrow$	<code>\iff</code>	$\leq$	<code>\le</code>	<code>\le</code>	<code>\le</code>	$\alpha \beta$	<code>\a \b</code>

⚠ Attenzione

Alcuni simboli hanno un'alternativa ASCII (`->` per $\rightarrow$, **fun** per λ , `<=` per $\leq$), la maggior parte no. E attenzione: **!= non** è $\neq$ — è **bne**, la disuguaglianza booleana.

Dove cercare

PER IMPARARE

- ▶ *Theorem Proving in Lean 4* — il manuale, dalle basi;
- ▶ Avigad e Massot, *Mathematics in Lean* — matematica vera, con esercizi;
- ▶ *Natural Number Game*
<https://adam.math.hhu.de>

PER ANDARE OLTRE MATHLIB

CSLib — componenti riusabili per l'informatica: modelli di calcolo, complessità.
<https://www.cslib.io>

Tutti raccolti anche nel **README.md** del repo, con il **CHEATSHEET.md**.

PER CERCARE UN LEMMA

- ▶ documentazione di Mathlib
https://leanprover-community.github.io/mathlib4_docs/
- ▶ **Loogle** — per *forma*
<https://loogle.lean-lang.org>
- ▶ **LeanSearch** — in inglese
<https://leansearch.net>

PER CHIEDERE

Zulip:

<https://leanprover.zulipchat.com>

Prima di cominciare: il setup

Opzione A — locale

- ▶ `elan` + VS Code + estensione Lean 4;
- ▶ progetto Mathlib **già compilato**;
- ▶ `lake exe cache get` scarica GB: **va fatto prima di arrivare**.

Opzione B — zero install

- ▶ GitHub Codespaces, dal repo: *Code* → *Codespaces*;
- ▶ <https://live.lean-lang.org> — ha Mathlib, ma è lento e non salva il lavoro.

Repository del corso

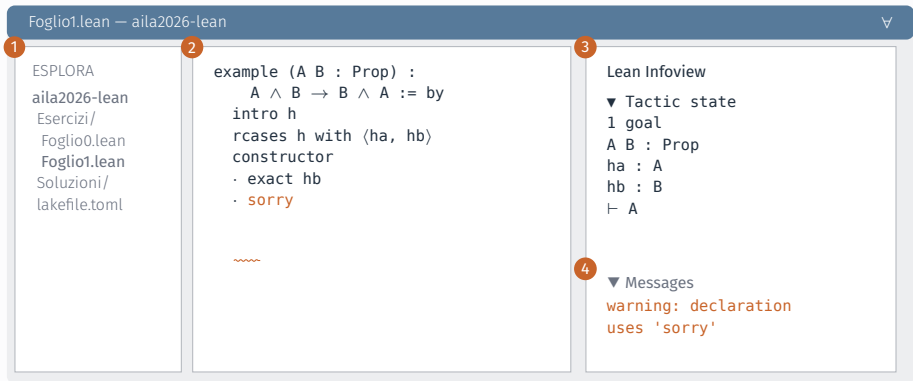
<https://github.com/miculan/aila2026-lean>

Cinque fogli in **Esercizi/**, con i **sorry** da riempire; le soluzioni commentate in **Soluzioni/**; un **CHEATSHEET.md** con tattiche ed errori comuni.

Blocco 1

Tour dell'ambiente — Foglio0

L'editor: che cosa si vede



1 apri la **cartella**, non il file

3 l'icona **∇** apre l'**Infoview**: il goal sotto il cursore

2 rosso = errore, giallo = **sorry**

4 output di **#check** e **#eval**

```
import Mathlib.Tactic

#check (2 + 2 : Nat)  -- 2 + 2 : Nat
#eval 2 + 2           -- 4

/- term mode: la dimostrazione È il termine -/
example (A : Prop) (h : A) : A := h

/- tactic mode: costruiamo il termine per passi -/
example (A : Prop) (h : A) : A := by
  exact h
```

- **Infview** mostra il *goal*: ipotesi sopra la barra, tesi sotto — è un contesto Γ e un tipo A ;
- **theorem/lemma** nominano il risultato, **example** no.

Come si legge un goal

```
A B C : Prop  
h1 : A → B  
h2 : B → C  
a : A
```

$\vdash C$



$\underbrace{A, B, C : \text{Prop}, h_1 : A \rightarrow B, h_2 : B \rightarrow C, a : A}_{\Gamma} \vdash ? : C$

La lista di ipotesi è il contesto Γ , la barra è il tornello, e il $?$ è il termine di prova che stiamo costruendo. [!\[\]\(d66ff64371a51729ac8c1cdaa685ba6f_img.jpg\) L1: lo stesso giudizio \$\Gamma \vdash t : A\$](#)

Blocco 2

Logica proposizionale come esercizio Curry–Howard — Foglio 1

L'esempio canonico, di nuovo

 L1: lo stesso termine costruito alla lavagna

```
-- term mode: il  $\lambda$ -termine, identico a quello di L1
example (A B C : Prop) (h1 : A  $\rightarrow$  B) (h2 : B  $\rightarrow$  C) : A  $\rightarrow$  C :=
  fun a => h2 (h1 a)

-- tactic mode: gli stessi passi, dall'altro verso
example (A B C : Prop) (h1 : A  $\rightarrow$  B) (h2 : B  $\rightarrow$  C) : A  $\rightarrow$  C := by
  intro a      --  $\rightarrow$ -intro : sposta A nel contesto
  exact h2 (h1 a)
```

Intuizione

intro è $\rightarrow$ I. **exact**/**apply** sono $\rightarrow$ E. Le tattiche costruiscono il termine di prova: **#print** lo mostra.

tattica	regola
intro	$\rightarrow I, \Pi I$
exact	chiude il goal
apply	$\rightarrow E$ all'indietro
constructor	$\wedge I$
left/right	$\vee I_{1,2}$
cases/rcases	$\vee E, \wedge E$
exfalso	$\perp E$

```
example (A B : Prop) : A  $\wedge$  B  $\rightarrow$  B  $\wedge$  A
  := by
  intro h
  rcases h with ⟨ha, hb⟩
  constructor
  · exact hb
  · exact ha
```

```
example (A B : Prop) : A  $\vee$  B  $\rightarrow$  B  $\vee$  A
  := by
  rintro (ha | hb)
  · right; exact ha
  · left; exact hb
```

L'esperimento: $\neg\neg A \rightarrow A$

```
example (A : Prop) :  $\neg\neg A \rightarrow A$  := by
  intro h
  --  $\vdash A$  , con  $h : \neg\neg A$  . Provare: intro? exact? apply h?
  sorry
```

⚠ Non si chiude

Costruttivamente non c'è modo: BHK non fornisce alcun metodo per estrarre una dimostrazione di A da $(A \rightarrow \perp) \rightarrow \perp$. [🔗](#) L1: ecco che cosa significava, concretamente

```
example (A : Prop) :  $\neg\neg A \rightarrow A$  := by
  intro h
  by_contra hA          -- Classical.byContradiction
  exact h hA
```

Che cosa abbiamo appena fatto, e a che prezzo

- ▶ **by_contra non** è una tattica come le altre: fa passare la prova per `Classical.em`
 : *stamattina: un teorema, non un assioma*;
- ▶ `#print axioms` lo rende visibile, ed è tutto il punto: la versione con **by_contra** elenca `Classical.choice`, l'altra no.

Che sia *lecito* lo sappiamo da ieri: il modello in ZFC  L3: *Carneiro, Werner*.

⚠ Il prezzo non è la coerenza: è la canonicità

Un termine chiuso di tipo $\mathbb{N}$ costruito con la scelta può non ridursi a un numerale: cadono disgiunzione ed esistenza, e da $\exists x. P(x)$ non si estrae più un testimone. In Lean quelle definizioni si marciano **noncomputable**.

✂ Esercizio

Dimostrare $A \rightarrow \neg\neg A$ e $\neg\neg\neg A \rightarrow \neg A$, poi `#print axioms`: non useranno la scelta.

Blocco 3

Tipi induttivi e induzione — Foglio2

$\mathbb{N}$ come tipo induttivo

```
inductive MyNat where  
  | zero : MyNat  
  | succ : MyNat → MyNat  
  
#check @MyNat.rec    -- l'eliminatore: È il principio di induzione
```

🔗 L3: $\text{ind}_{\mathbb{N}}$, scritto alla lavagna: è letteralmente questo

```
def add : Nat → Nat → Nat  
  | n, 0      => n                -- ricorsione sul SECONDO argomento  
  | n, .succ m => .succ (add n m)
```


$n + 0$ contro $0 + n$: la differenza di L3

```
-- definizionale: il kernel calcola, non serve dimostrare nulla
theorem add_zero_nat (n : Nat) : n + 0 = n := rfl

-- proposizionale: serve induzione
theorem zero_add_nat (n : Nat) : 0 + n = n := by
  induction n with
  | zero      => rfl
  | succ n ih => simp [Nat.add_succ, ih]
```

💡 Perché l'asimmetria

$+$ è definito per ricorsione sul *secondo* argomento: quindi $n + 0$ si riduce a n per β/ι , cioè $n + 0 \equiv n$; mentre $0 + n$ è bloccato finché n è una variabile. [🔗 L3](#): $\equiv$ vs Id

Esercizio guidato: commutatività

```
theorem my_add_comm (m n : Nat) : m + n = n + m := by  
  induction n with  
    | zero      => simp  
    | succ n ih => rw [Nat.add_succ, ih, Nat.succ_add]
```

Somma di Gauss

```
theorem gauss (n : Nat) : 2 * ( $\sum i \in \text{Finset.range } (n+1), i$ ) = n * (n+1) := by  
  sorry
```

✂ In autonomia o in coppia

1. $(A \rightarrow B \rightarrow C) \rightarrow (A \wedge B \rightarrow C)$ e il viceversa (*currying*);
2. le leggi di De Morgan: quali valgono costruttivamente?
3. `length (l ++ l') = length l + length l'` su **List**;
4. `reverse (reverse l) = l`.

📌 Nota

Sono nel **Foglio2.lean**, insieme agli altri. Le soluzioni stanno in **Soluzioni/** — da aprire *dopo* averci provato. Le rivediamo a inizio L5.

- ▶ J. Avigad e P. Massot. *Mathematics in Lean*. Documentazione online, aggiornata di continuo. 2025. URL: https://leanprover-community.github.io/mathematics_in_lean/ — capitoli 1–3.
- ▶ J. Avigad et al. *Theorem Proving in Lean 4*. 2025. URL: https://leanprover.github.io/theorem_proving_in_lean4/
- ▶ L. de Moura e S. Ullrich. «The Lean 4 Theorem Prover and Programming Language». In: *Automated Deduction (CADE-28)*. Vol. 12699. LNCS. Springer, 2021, pp. 625–635
- ▶ M. Carneiro. «The Type Theory of Lean». Tesi di laurea mag. Carnegie Mellon University, 2019 — il modello che giustifica gli assiomi.

LEZIONE 5

Laboratorio Lean II: strutture algebriche elementari e Mathlib

AILA 2026 — Bardonecchia

Obiettivo

Passare da «logica al calcolatore» a **matematica al calcolatore**, e dare un primo assaggio della scala industriale di Mathlib.

Ripasso: gli esercizi di ieri

LE QUATTRO LEGGI DI DE MORGAN — quali valgono costruttivamente?

$$\neg(A \vee B) \rightarrow \neg A \wedge \neg B \quad \checkmark$$

$$\neg A \vee \neg B \rightarrow \neg(A \wedge B) \quad \checkmark$$

$$\neg A \wedge \neg B \rightarrow \neg(A \vee B) \quad \checkmark$$

$$\neg(A \wedge B) \rightarrow \neg A \vee \neg B \quad \times$$

⚠ L'unica che manca non è un caso

Sopra la logica intuizionista è **equivalente** al terzo escluso debole, $\neg A \vee \neg\neg A$: assumerla significa già assumere un frammento di logica classica.

```
example (A B : Prop) :  $\neg(A \wedge B) \rightarrow \neg A \vee \neg B$  := by
  intro h
  by_cases hA : A
    -- qui, e solo qui, entra Classical
    · right; intro hb; exact h ⟨hA, ⟩hb
    · left; exact hA
```

Currying e i due esercizi su [List](#): in `Soluzioni/Foglio2.lean`.

Blocco 1

Formalizzare gruppi e monoidi da zero

Una struttura algebrica è una `class`

```
class MyGroup (G : Type) extends Mul G, One G, Inv G where  
  mul_assoc :  $\forall a\ b\ c : G, (a * b) * c = a * (b * c)$   
  one_mul    :  $\forall a : G, 1 * a = a$   
  mul_one    :  $\forall a : G, a * 1 = a$   
  inv_mul    :  $\forall a : G, a^{-1} * a = 1$ 
```

- ▶ una **class** è una **structure** con *risoluzione automatica delle istanze*:
[MyGroup G] è un argomento che Lean cerca da solo;
- ▶ **extends** realizza l'ereditarietà multipla della gerarchia algebrica;
- ▶ i campi sono *proposizioni*: la struttura porta con sé le sue dimostrazioni.  L3: Σ -tipi

Il primo teorema: unicità dell'inverso

```
theorem inv_unique {G : Type} [MyGroup G] (a b : G) (h : a * b = 1) :  
  b = a-1 := by  
  calc b = 1 * b           := (MyGroup.one_mul b).symm  
    _ = (a-1 * a) * b      := by rw [MyGroup.inv_mul]  
    _ = a-1 * (a * b)      := MyGroup.mul_assoc _ _ _  
    _ = a-1 * 1           := by rw [h]  
    _ = a-1               := MyGroup.mul_one _
```

Il blocco `calc`

È il modo in cui una catena di uguaglianze «da carta» si scrive in Lean restando leggibile. Ogni passo va giustificato: nessuna scorciatoia, ma il risultato somiglia a una dimostrazione scritta a mano.

Esercizio

1. unicità dell'elemento neutro: se $\forall a. e * a = a$ allora $e = 1$;
2. legge di cancellazione: $a * b = a * c \Rightarrow b = c$;
3. $(a * b)^{-1} = b^{-1} * a^{-1}$;
4. $(a^{-1})^{-1} = a$.

Perché proprio questi

Sono brevi ma **non** si chiudono con **simp** a caso: costringono a ragionare su *quale* assioma serve e in quale direzione riscrivere. È l'analogo formale di un esercizio da primo corso di algebra.

Automazione: che cosa fa davvero

simp	riscrittura (<i>simp set</i>)
ring	anelli commutativi
group	gruppi
omega	aritmetica lineare su $\mathbb{N}/\mathbb{Z}$
decide	predicati decidibili
linarith	disuguaglianze lineari

```
example (x y :  $\mathbb{Z}$ ) : (x + y)^2 = x^2 +  
2*x*y + y^2 := by
```

```
ring
```

```
example (n : Nat) (h : n > 3) : n ≥ 4 :=  
by  
omega
```

```
example : Nat.Prime 97 := by  
decide
```

Attenzione

Tutte producono un **termine di prova** che passa dal kernel  *L1: de Bruijn criterion*. Ma nessuna sostituisce la comprensione degli assiomi: quando falliscono, bisogna sapere perché.

Blocco 2

Mathlib e il «mondo reale»

I nomi di Mathlib si *leggono*

Il nome di un lemma **descrive l'enunciato**, letto da sinistra a destra. Imparata la convenzione, metà della ricerca è fatta.

<code>mul add sub</code>	<code>*</code> <code>+</code> <code>-</code>
<code>neg inv</code>	<code>-a</code> <code>a⁻¹</code>
<code>le lt ne</code>	<code>≤</code> <code><</code> <code>≠</code>
<code>zero one</code>	<code>0</code> <code>1</code>
<code>nonneg pos</code>	<code>0 ≤ a</code> <code>0 < a</code>
<code>_of_</code>	conclusione $\leftarrow$ ipotesi
<code>_iff_</code>	$\leftrightarrow$

```
mul_comm   : a * b = b * a
one_mul    : 1 * a = a
le_of_lt   : a < b → a ≤ b
add_le_add_left :
  b ≤ c → ∀ a, a + b ≤ a + c
sub_nonpos : a - b ≤ 0 ↔ a ≤ b
```

L'apice ' segna una variante dello stesso enunciato. E `#check @mul_comm` mostra sempre la forma completa, con argomenti impliciti e istanze.

Gli strumenti di ricerca

DAL GOAL — tattiche

exact?	chiude il goal, se può
apply?	elenca i lemmi applicabili
rw?	propone riscritture
simp?	mostra che cosa ha usato
hint	prova più tattiche insieme

simp? è il più educativo: sostituisce **simp** con la lista esplicita dei lemmi, e la prova diventa stabile agli aggiornamenti di Mathlib.

Nota

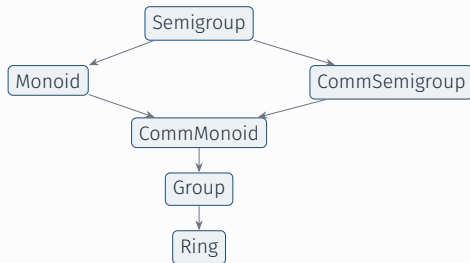
~2M righe, oltre 500 contributori: a questa scala la difficoltà non è dimostrare, è **trovare**.  L6: ed è qui che entra il premise selection con l'IA

DALLA FORMA — motori esterni

```
-- Loogle: cerca per forma
|- _ * _ = _ * _
Nat.succ, (_ + _)
"comm"
```

Loogle cerca per *forma* dell'enunciato, per costanti che vi compaiono o per sottostringa del nome; **LeanSearch** accetta l'inglese.

La gerarchia algebrica



Ogni struttura è una **class**; l'ereditarietà è **multipla**, e la risoluzione delle istanze percorre il grafo da sola.

Scrivendo **[Ring R]** si ottengono gratis *tutti* i teoremi su semigrupp, monoidi, gruppi abeliani... centinaia, senza riscrivere nulla.

È il motivo per cui vale la pena formalizzare dentro Mathlib invece che accanto: si eredita, non si ricomincia.

⚠ Ma i cammini si incontrano

A **CommMonoid** si arriva da **Monoid** e da **CommSemigroup**. Che cosa garantisce che le due strade diano la stessa moltiplicazione?

Il diamond problem

Il problema non è che le due istanze siano *diverse*: è che devono essere **definizionalmente** uguali, non solo proposizionalmente. 🔗 L3: *di nuovo* $\equiv$ *contro* *Id*

Se non lo sono, **rw** e **simp** falliscono con errori oscuri — il classico *motive is not type correct* — e due lemmi che parlano dello stesso oggetto non si compongono.

LE DUE CONTROMISURE DI MATHLIB

- ▶ **strutture piatte**: **extends** copia *tutti* i campi del genitore nel figlio, così esiste un solo cammino canonico ai dati;
- ▶ **campi ridondanti**: una struttura porta con sé operazioni che si potrebbero derivare. **Ring** ha un campo **sub**, con l'assioma **sub_eq_add_neg**, invece di definire $a - b := a + (-b)$.

Senza il secondo trucco, la sottrazione *nativa* di $\mathbb{Z}$ non sarebbe definizionalmente quella derivata, e ogni lemma sulla sottrazione andrebbe dimostrato due volte.

PFR

Congettura Polynomial Freiman–Ruzsa. Tao *et al.*, 2023: formalizzata in Lean in ~ 3 settimane, in modo collaborativo e distribuito (*blueprint* + decine di contributori).

PNT forte

Teorema dei numeri primi, versione forte: progetto fermo 18+ mesi, completato in ~ 3 settimane con l'aiuto di un sistema di IA (Math Inc. / «Gauss»), su una sfida di Tao e Kontorovich.

Sphere packing

La dimostrazione di Viazovska (Medaglia Fields) per l'impacchettamento ottimo in dimensione 24, formalizzata in Lean in circa due settimane.

Intuizione

Il collo di bottiglia **non** era trovare la matematica: era tradurla in forma verificabile.

 L6: è lì che l'IA sta avendo impatto

✂ Da portare avanti durante la scuola

1. definire **MyMonoid** e dimostrare l'unicità del neutro;
2. definire un omomorfismo di monoidi e dimostrare che la composizione di omomorfismi è un omomorfismo;
3. un anello elementare: $0 \cdot a = 0$ a partire dagli assiomi;
4. (facoltativo) rifare (1) usando le classi di Mathlib invece delle proprie, e confrontare.

- ▶ J. Avigad e P. Massot. *Mathematics in Lean*. Documentazione online, aggiornata di continuo. 2025. URL: https://leanprover-community.github.io/mathematics_in_lean/
- ▶ The mathlib Community. «The Lean Mathematical Library». In: *Certified Programs and Proofs (CPP 2020)*. ACM, 2020, pp. 367–381
- ▶ T. Tao, Y. Dillies et al. *A Formalisation of the Polynomial Freiman–Ruzsa Conjecture*. Progetto di formalizzazione collaborativa in Lean 4. 2023. URL: <https://github.com/teorth/pfr>
- ▶ R. Affeldt et al. «Competing Inheritance Paths in Dependent Type Theory: A Case Study in Functional Analysis». In: *Automated Reasoning (IJCAR 2020)*. Vol. 12167. LNCS. Springer, 2020, pp. 3–20 — sul *diamond problem*.

LEZIONE 6

Proof assistant e Intelligenza Artificiale: verso una nuova pratica della matematica

AILA 2026 — Bardonecchia

Obiettivo

Chiudere il cerchio del corso mostrando che il tema non è futuristico ma **già operativo**, e aprire una discussione su che cosa cambia epistemologicamente nella pratica matematica.

6. Lezione 6 — Proof assistant e IA

6.1 Due problemi da tenere distinti

6.2 Chi propone e chi valida

6.3 La divisione del lavoro

6.4 Architetture

6.5 Come si misura il progresso

6.6 Discussione

Che cosa abbiamo costruito, e la domanda di oggi

In cinque lezioni abbiamo costruito una macchina che **controlla** una dimostrazione, con la fiducia concentrata in un kernel piccolo e ispezionabile  *L1–L5: dalla teoria al laboratorio.*

Ma adesso le dimostrazioni cominciano a scriverle anche le macchine.

- ▶ **Che cosa sanno fare davvero**, e che cosa si riesce a tenere sotto controllo? blocco 1
- ▶ **Che cosa cambia** nella pratica della matematica — e nella nozione stessa di dimostrazione? blocco 2

Nota

Il corso si è aperto su dimostrazioni troppo grandi perché una mente le controlli 
L0: la crisi della complessità. Si chiude chiedendosi che cosa succede quando a scriverle non è più solo una mente. (È l'unica lezione in cui non dimostreremo niente.)

Blocco 1

Panorama tecnico

Autoformalizzazione

linguaggio naturale $\rightarrow$ enunciato
(e/o dimostrazione) Lean

⚠ La correttezza dell'enunciato **non** è verificabile dal kernel: è una questione di *fedeltà*, non di validità.

Proof search

enunciato *già formale* $\rightarrow$ dimostrazione Lean

✓ Verificabile al 100%: o il kernel accetta, o no. Il modello può sbagliare quanto vuole.

💡 Intuizione

Il **de Bruijn criterion** di L1 è ciò che rende la seconda colonna innocua e la prima l'unico vero punto di fiducia residuo. Tutta la discussione epistemologica di oggi vive in quella asimmetria.

Il limite da cui parte tutto: l'allucinazione

Un LLM **inventa** teoremi falsi, e li enuncia con la stessa sicurezza di quelli veri.

I would caution against using AI tools without the ability to independently verify their output. Relying on these tools to compensate for their own mistakes is quite risky and can amplify the weaknesses.

— Terence Tao

La risposta non è «un modello migliore»: è un canale di verifica *indipendente* dal modello. È esattamente ciò che un proof assistant sa fare — e il motivo per cui questa lezione sta in un corso di teoria della dimostrazione.

Non è un problema dell'IA: gli strumenti sbagliano già

Prima ancora dei modelli generativi, il software matematico che usiamo ogni giorno produce risposte sbagliate — e in silenzio.

```
-- SymPy 1.5.1: due strade per la stessa espressione, due risultati diversi
simplify(hyper([n],[m],x).subs({m:-1, n:-1, x:1}))
simplify(hyper([n],[m],x).subs(m, n)).subs({n:-1, x:1})
```

Mathematica esibisce lo stesso fenomeno; e in FLINT è stato trovato un bound errato per $\sqrt{x}$.

⚠ La conclusione di Mahboubi

«*Cross-verification is not enough*»: confrontare due strumenti non basta, se sbagliano allo stesso modo. E oggi non esiste alcuna politica esplicita per certificare il software che produce passi di dimostrazione. 🔗 : *Computer-Assisted Mathematics, TYPES 2026*

Chi propone non può essere chi valida

Il principio: **generazione e verifica vanno disaccoppiate**. Un generatore incerto propone, un verificatore separato impone i vincoli, si esegue solo ciò che passa. [Mic26]

E CIÒ CHE CONTA È IL FALLIMENTO *CONGIUNTO* q , NON L'ERRORE DEL SINGOLO

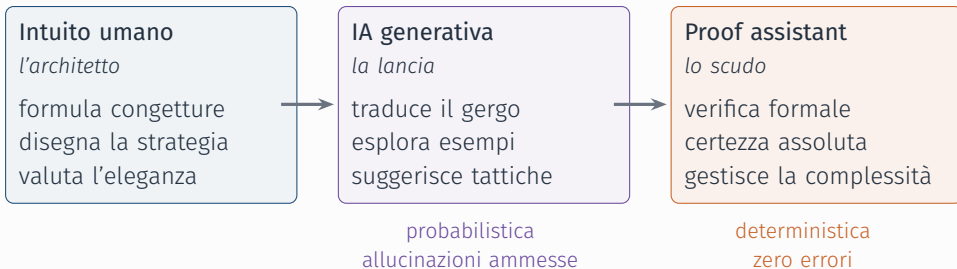
generatore + controllore	q	$R = 1 - q$
due LLM dello stesso fornitore	0,067	0,933
due LLM di fornitori <i>diversi</i> (caso peggiore)	0,133	0,867
LLM + verificatore formale	0,000	1,000

Misurato su Blocksworld, cinque LLM. **La diversità va misurata, non dedotta dall'etichetta del fornitore.**

⚠ E questo lo conosciamo già

Con un verificatore *corretto* l'azione eseguita è ammissibile **qualunque cosa faccia il generatore**: è il *de Bruijn criterion* di L1, come principio di architettura.

L'umano immagina, l'IA propone, il proof assistant dispone



💡 Intuizione

La divisione del lavoro non è un compromesso: è ciò che rende *utilizzabile* uno strumento che sbaglia. L'IA può permettersi di sbagliare perché a valle c'è qualcosa che non sbaglia mai 🔗 *L1: de Bruijn criterion.*

Cinque famiglie di sistemi

Approccio	Idea	Esempi
Copilota interattivo	suggerisce la prossima tattica dentro il workflow dell'utente	Lean Copilot, LLMStep
Agente end-to-end	RL su ricerca di dimostrazioni, decomposizione in sottoobiettivi	DeepSeek-Prover, Goedel-Prover, Kimina-Prover
Retrieval di premesse	selezionare i lemmi Mathlib rilevanti  <i>L5: il vero collo di bottiglia</i>	LeanDojo, ReProver
Draft-sketch-prove	usare una dimostrazione informale come scaletta di quella formale	DSP e successori
LLM generalista in loop col compilatore	nessun addestramento su Lean: il modello scrive, legge gli errori, riprova — spesso dentro un agente di <i>coding</i>	Claude, GPT, Gemini

L'ultima è la più recente e la meno studiata: i confronti diretti fra famiglie vanno presi con le molle.

Il ciclo di un agente dimostratore



i Nota

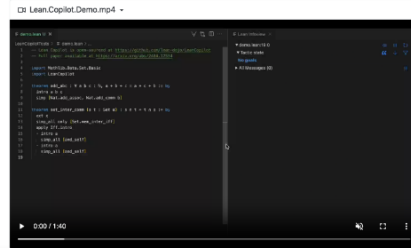
Il kernel non è solo il garante finale: è anche il **segnale di ricompensa**. È ciò che rende questo dominio così adatto al reinforcement learning — a differenza della matematica in linguaggio naturale.

Lean Copilot: l'IA dentro l'editor

- ▶ modelli linguistici integrati **dentro** Lean, non accanto: il suggerimento arriva nell'Infoview  L4: *dove già guardiamo il goal*;
- ▶ l'IA propone la **tattica** successiva; il matematico approva o scarta — e ciò che approva passa comunque dal kernel;
- ▶ **premise selection**: trova in un istante i lemmi di Mathlib rilevanti per *questo* goal  L5: *il vero collo di bottiglia*.

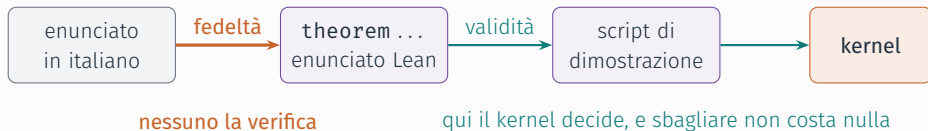
Lean Copilot: LLMs as Copilots for Theorem Proving in Lean

Lean Copilot allows large language models (LLMs) to be used in Lean for proof automation, e.g., suggesting tactics/premises and searching for proofs. You can use our built-in models from [LeanDojo](#) or bring your own models that run either locally (w/ or w/o GPUs) or on the cloud.



Il salto rispetto a un chatbot non è la qualità del suggerimento: è che il suggerimento nasce già dentro il contesto formale, e viene scartato automaticamente se non tipa.

Autoformalizzare con un agente



L'agente fa *tutto* il percorso: traduce, cerca il lemma, scrive, legge l'errore del compilatore, riprova. Ma le due frecce non sono uguali 🔗: è *l'asimmetria della prima slide di oggi*.

⚠ Il protocollo, quando si lavora così

Leggere l'enunciato prima della dimostrazione: se è sbagliato quello, una prova che compila non vale nulla. Poi `#print axioms`. E se chiude *troppo* in fretta, sospettare: spesso vuol dire che l'enunciato è diventato banale 🔗: *ci torniamo in blocco 2*.

Il fattore di de Bruijn, e il suo crollo

Quanto costa formalizzare una dimostrazione già scritta? Si misura con il **fattore di de Bruijn**: il rapporto fra lo sforzo formale e quello informale.

2023–24

~ 20×

2025–26

molti passi formalizzati *in tempo reale*

⚠ Che cosa cambia davvero

Finché formalizzare costa venti volte dimostrare, lo si fa solo per i teoremi che contano. Quando il costo crolla, diventa un'abitudine — e cambia *che cosa* si decide di formalizzare.

- ▶ **miniF2F** — olimpiadi scolastiche, formalizzate;
- ▶ **ProofNet** — esercizi da corsi universitari;
- ▶ **PutnamBench** — problemi della Putnam Competition;
- ▶ **FrontierMath** — problemi originali di ricerca, non formalizzati.

We have sort of selectively disclosed results, which look impressive, but we don't know exactly what the prompts were. We don't know the failure rate. And many of the companies that are disclosing these have their own incentives to maybe present the results in as favourable [a] light as possible.

— Terence Tao, ICM 2026

Di qui **First Proof**: problemi *non pubblicati*, così i modelli non possono ripescare la risposta. Su dieci posti, sette soluzioni sono state giudicate pubblicabili.

2024

IMO 2024

AlphaProof e AlphaGeometry-2 (DeepMind): livello **medaglia d'argento**, con dimostrazioni in Lean.

2025

IMO 2025

Aristotle (Harmonic) e Seed-Prover (ByteDance): livello **medaglia d'oro**, con dimostrazioni *verificate in Lean*.

OpenAI e DeepMind: stesso livello, ma in linguaggio naturale.

2026

Oltre le olimpiadi

Scoperta semi-autonoma su problemi *aperti* della lista di Erdős (Gemini, DeepMind).

La distinzione «verificato in Lean» contro «linguaggio naturale valutato da umani» è la stessa asimmetria della prima slide del blocco.

Blocco 2

Casi di studio e discussione epistemologica

I tre casi, riletti

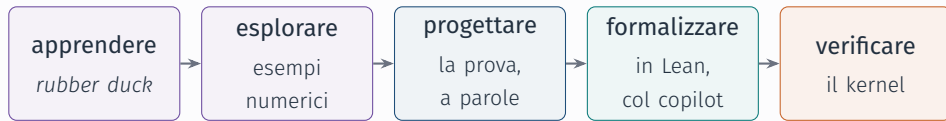
🔗 L5: li abbiamo già incontrati: ora chiediamoci che cosa mostrano

- ▶ **PFR**: la formalizzazione su larga scala diventa un'attività di progetto, con un *blueprint* che scompone il lavoro — un'infrastruttura sociale, prima che tecnica;
- ▶ **PNT forte**: 18 mesi di stallo → 3 settimane. Non è stata l'IA a trovare la matematica: era già nota. Ha sbloccato la *traduzione*;
- ▶ **Sphere packing in dim. 24**: una dimostrazione da Medaglia Fields, con analisi pesante, formalizzata in settimane.

💡 Intuizione

Il collo di bottiglia si è spostato da «trovare la dimostrazione» a «tradurla in forma meccanicamente verificabile».

Il nuovo workflow di Terence Tao



- ▶ l'IA entra dove il lavoro è *ripetitivo o esplorativo*, non dove è creativo: il passo centrale — disegnare la struttura della prova — resta in linguaggio naturale, e resta umano;
- ▶ «rubber duck»: farsi spiegare da un modello un settore che non si conosce, sapendo che può sbagliare, perché serve a *orientarsi*, non a citare.

Dalla matematica artigianale a quella industriale

ARTIGIANALE

un autore, un articolo, una dimostrazione letta da pochi specialisti che se ne fanno garanti.

INDUSTRIALE

grandi collaborazioni, librerie di codice riusabile, verifica automatica di ogni pezzo.

You don't need to trust the people you're working with, because the program gives this 100 percent guarantee.

— Terence Tao

- ▶ cade il **principio di autorità**: non conta chi firma, conta se compila — ed è ciò che ha reso possibile PFR con venti collaboratori in tre settimane  L5: *ce lo siamo già incontrato*;
- ▶ nascono ruoli nuovi: chi formalizza, chi *digerisce* le prove altrui, e la «citizen mathematics» di studenti e dilettanti.

- ▶ **scoperta semi-autonoma**: problemi aperti della lista di Erdős affrontati con modelli Gemini (DeepMind, 2026);
- ▶ **autoformalizzazione di interi libri di testo**;
- ▶ **librerie di dominio** che estendono Mathlib oltre la matematica pura: *CSLib* per l'informatica;
- ▶ **verifica industriale**: la specifica formale del linguaggio di autorizzazione Cedar di AWS è scritta *direttamente* in Lean — valutatore, autorizzatore e validatore verificati.

Nota

La stessa infrastruttura che verifica il teorema dei numeri primi verifica un motore di controllo degli accessi in produzione.

ALEX KONTOROVICH — Rutgers

Nell'advisory board di Lean, plenaria all'ICM sull'autoformalizzazione. La sua motivazione è disarmante:

Posso fare quel calcolo a mano cinque volte finché non ottengo due volte lo stesso risultato, oppure lo do a una macchina — e non perdo mai più tempo a ricontrollarlo.

ROBERT GHRIST — Pennsylvania

L'IA come **ponte fra domini**: ha chiesto a un agente dove applicare una certa astrazione, si è sentito rispondere «reti finanziarie», ha passato un anno a studiarle e ha pubblicato.

AI is potentially going to lead to a real renaissance in applied math.

Due usi che non compaiono in nessun benchmark: togliere di mezzo il lavoro di controllo, e far parlare fra loro settori che non si parlavano.

L'impiego più affidabile dell'autoformalizzazione, oggi, non è *dimostrare*: è **scovare enunciati sbagliati**.

Il segnale

Quando un modello formalizza e chiude una serie di enunciati *troppo* facilmente, di solito non è bravo: è che l'enunciato è stato tradotto male, ed è diventato banale. La facilità improvvisa è un campanello d'allarme, e rende un archivio *scandagliabile* alla ricerca di affermazioni false.

È l'unico uso in cui l'allucinazione non è un problema: qui il modello non deve avere ragione, deve solo far emergere una discrepanza fra l'enunciato informale e quello formale  : *la fedeltà della prima slide*.

Da scarsità ad abbondanza di dimostrazioni

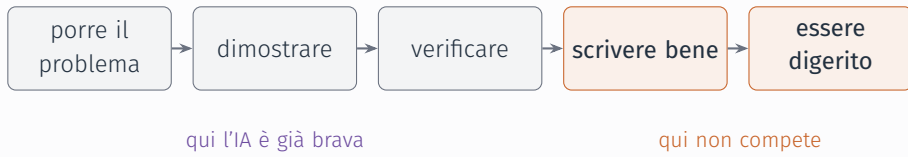
Nel 2026 problemi rimasti aperti per decenni cadono al ritmo di **qualche settimana**, a volte per mano di dilettanti che si sono limitati a chiedere a un modello.

💡 La metafora di Tao

I matematici guidavano su strade costruite per pedoni e cavalli. Ora è arrivata l'IA, con auto più grandi e più veloci: gli ingorghi sono inevitabili, e l'infrastruttura mostra tutte le sue crepe. Generare dimostrazioni era il collo di bottiglia; ora lo sono la revisione e l'accettazione. «*Validation is in short supply.*»

L'ultima crisi seria della matematica è dei primi del Novecento, quando i paradossi minacciarono le fondazioni — ed è la crisi da cui nasce tutto ciò che abbiamo visto in questo corso 🔗 L2-L3: *teoria dei tipi*. «That was traumatic», dice Tao. Questa è diversa: non riguarda gli argomenti matematici, ma **valori e pratiche**.

Che cosa fa davvero un matematico



In order to actually influence the future development of mathematics, it needs to be accepted and valued by [the] community. Other mathematicians need to actually want to read it and digest it and put it into their own work.

— Terence Tao, ICM 2026

E le dimostrazioni generate dall'IA sono *frustranti* da leggere: pagine su ciò che è ovvio, poche righe sul passaggio difficile.

I matematici hanno perso la narrazione

Until recently, only mathematicians cared about mathematics. We had full control. Now, for the first time, mathematicians have temporarily lost the narrative.

— Terence Tao

- ▶ l'equivoco che si sta affermando: che la matematica sia *soprattutto* risolvere problemi aperti, con qualunque mezzo — «maths as a sport where you just collect tick marks»;
- ▶ la sua richiesta non è difensiva: organizzarsi, farsi sentire, occuparsi anche della politica della disciplina.

They tried to redefine what our profession is. [But] mathematicians have more influence than they think.

— Terence Tao

- ▶ **formazione:** Tao riferisce di studenti la cui abilità è declinata dopo un uso eccessivo dell'IA — «at some point, it becomes irreversible»;
- ▶ **comprensione:** il rischio non è la matematica sbagliata, è la matematica *funzionante ma opaca* — corretta, verificata, e che nessuno capisce;
- ▶ **risorse:** l'IA richiede potenza di calcolo enorme. Chi controllerà la direzione della ricerca, se gli strumenti sono di pochi?

You can use AI if you can present your work to your advisors and answer questions intelligently without looking at your phone.

— Terence Tao

I rischi, dal meno al più urgente secondo Tao: malfunzionamento autonomo $\ll$ uso umano scorretto $<$ disruption socioeconomica $<$ blocco per panico $<$ uso malevolo — e il primo è quello di cui si parla di più.

Che cosa si sta già facendo

- ▶ **Leiden Declaration on AI and Mathematics** — iniziativa dal basso, approvata dall'International Mathematical Union e firmata da oltre **3000** matematici;
- ▶ **First Proof** — valutare i modelli su problemi non pubblicati, per sapere davvero che cosa sanno fare;
- ▶ **Mathematical Discourse** — rivista in forma di video lanciata da Tao, dove un contributo si giudica anche da *quanto bene lo sai spiegare ai colleghi*, non solo dalla correttezza.

Mathematics produces not only a body of results, but also understanding, clarity, and judgment among the communities of mathematicians who have shaped them.

— Leiden Declaration, 2026

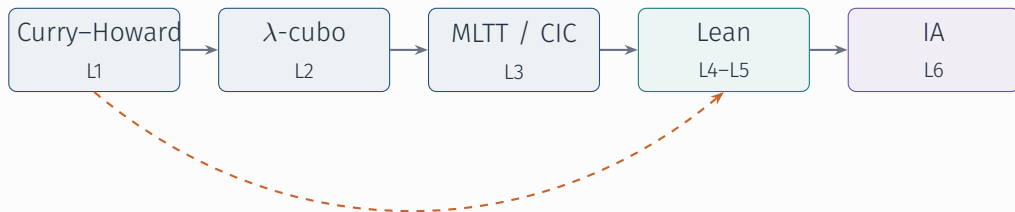
Tao, firmandola: «in retrospect, these were questions we should have been systematically discussing years ago».

Che cosa cambia
nella nozione di *dimostrazione*?

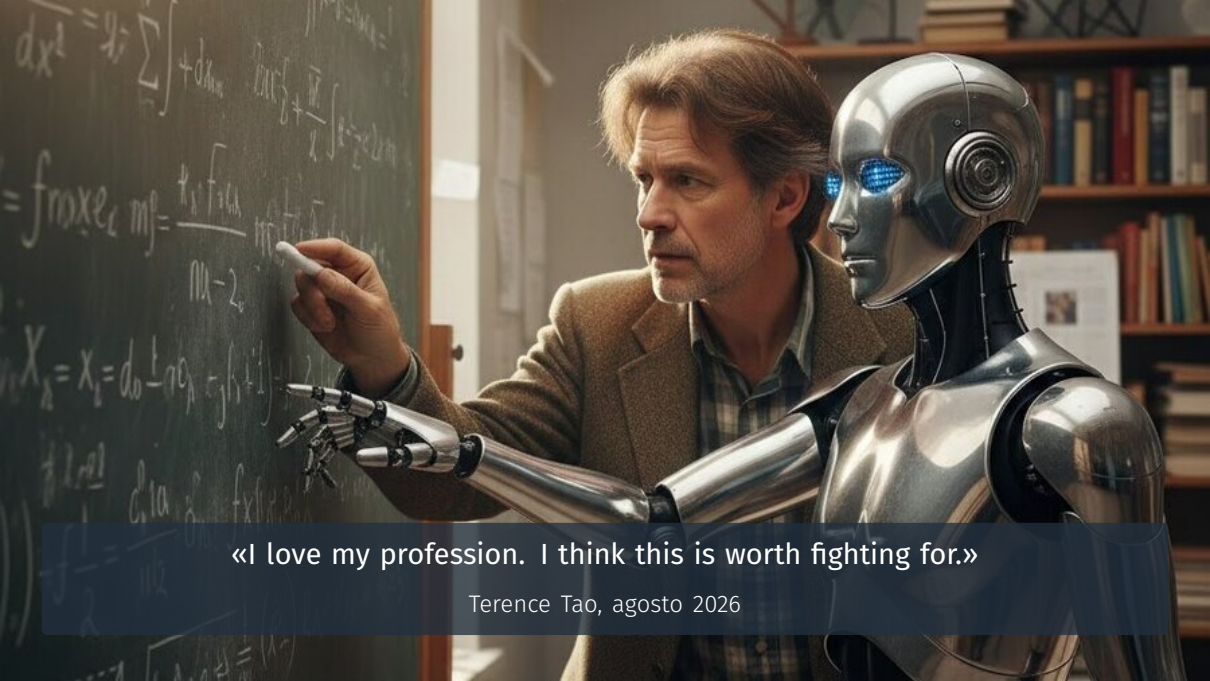
Quattro domande aperte

1. Se la verifica è meccanica al 100% ma la **scrittura** è parzialmente o totalmente automatica, che cosa resta dell'idea che una dimostrazione debba essere *comprensibile*?
2. Chi è responsabile di un errore che il kernel non può *in linea di principio* catturare — un enunciato formalizzato male, cioè non fedele all'enunciato informale?
3. Che cosa significa **autorship** per un teorema formalizzato da cinquanta persone e tre modelli?
4. Il **peer review** deve ancora leggere la dimostrazione, o solo l'enunciato formale?

Il percorso, dall'inizio



- ▶ una sola idea — *le dimostrazioni sono oggetti* — portata fino in fondo;
- ▶ il kernel piccolo e fidato di L1 è ciò che rende possibile il panorama di L6;
- ▶ per proseguire: *Mathematics in Lean*, il Lean Zulip, i progetti di formalizzazione aperti.



«I love my profession. I think this is worth fighting for.»

Terence Tao, agosto 2026

- ▶ K. Yang et al. «LeanDojo: Theorem Proving with Retrieval-Augmented Language Models». In: *Advances in Neural Information Processing Systems (NeurIPS 2023), Datasets and Benchmarks Track*. 2023
- ▶ P. Song, K. Yang e A. Anandkumar. *Towards Large Language Models as Copilots for Theorem Proving in Lean*. 2024. arXiv: 2404.12534 [cs.AI]
- ▶ Z. Z. Ren et al. *DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition*. 2025. arXiv: 2504.21801 [cs.CL]. URL: <https://arxiv.org/abs/2504.21801>
- ▶ T. Achim et al. *Aristotle: IMO-level Automated Theorem Proving*. 2025. arXiv: 2510.01346 [cs.AI]. URL: <https://arxiv.org/abs/2510.01346>

- ▶ T. Tao, Y. Dillies et al. *A Formalisation of the Polynomial Freiman–Ruzsa Conjecture*. Progetto di formalizzazione collaborativa in Lean 4. 2023. URL: <https://github.com/teorth/pfr>
- ▶ T. Feng et al. *Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems*. 2026. arXiv: 2601.22401 [cs.AI]. URL: <https://arxiv.org/abs/2601.22401>
- ▶ AWS Cedar Team. *cedar-spec: Formal Specification and Verification of the Cedar Authorization Language in Lean*. 2024. URL: <https://github.com/cedar-policy/cedar-spec>
- ▶ T. Tao. *Mathematics in the Age of AI*. Saggio basato sulla conferenza pubblica all'ICM 2026. 2026. arXiv: 2608.16753 [math.HO]

- ▶ K. Padavic-Callaghan. «**Why Mathematician Terence Tao Thinks AI Must Spark a Rapid Revolution**». In: *New Scientist* (ago. 2026). Intervista a Terence Tao all'ICM di Filadelfia, 6 agosto 2026
- ▶ T. Tao. *Terence Tao on AI in Mathematics (and Beyond)*. Sintesi aggiornata delle proprie posizioni; cfr. anche *Mathematical Discourse* e il progetto *First Proof*. 2026.
URL: <https://teorth.github.io/tao-web/ai-views.html>
- ▶ The Leiden Declaration on Artificial Intelligence and Mathematics. *Leiden Declaration on Artificial Intelligence and Mathematics*. Approvata dall'International Mathematical Union; oltre 3000 firmatari. 2026
- ▶ A. Mahboubi. *Computer-Assisted Mathematics*. Conferenza invitata, TYPES 2026, 4 maggio 2026. 2026

- ▶ M. Miculan. «**Reliable Decision Loops in Socio-Technical Collective Adaptive Systems**». In: *Leveraging Applications of Formal Methods, Verification and Validation (ISoLA 2026)*. LNCS. Track REoCAS — Rigorous Engineering of Collective Adaptive Systems. In corso di stampa. Springer, 2026
- ▶ Y. Duong. *AI Will Be Top of Mind at ICM, Math's Biggest Conference*. Simons Foundation, 4 maggio 2026. 2026. URL: <https://www.simonsfoundation.org/2026/05/04/ai-will-be-top-of-mind-at-icm-maths-biggest-conference/>

- [Ach+25] T. Achim et al. *Aristotle: IMO-level Automated Theorem Proving*. 2025. arXiv: [2510.01346](https://arxiv.org/abs/2510.01346) [cs.AI]. URL: <https://arxiv.org/abs/2510.01346>.
- [Acz78] P. Aczel. «The Type Theoretic Interpretation of Constructive Set Theory». In: *Logic Colloquium '77*. A cura di A. Macintyre, L. Pacholski e J. Paris. North-Holland, 1978, pp. 55–66.
- [Aff+20] R. Affeldt et al. «Competing Inheritance Paths in Dependent Type Theory: A Case Study in Functional Analysis». In: *Automated Reasoning (IJCAR 2020)*. Vol. 12167. LNCS. Springer, 2020, pp. 3–20.
- [AM25] J. Avigad e P. Massot. *Mathematics in Lean*. Documentazione online, aggiornata di continuo. 2025. URL: https://leanprover-community.github.io/mathematics_in_lean/.
- [Avi+25] J. Avigad, L. de Moura, S. Kong e S. Ullrich. *Theorem Proving in Lean 4*. 2025. URL: https://leanprover.github.io/theorem_proving_in_lean4/.

- [AWS24] AWS Cedar Team. *cedar-spec: Formal Specification and Verification of the Cedar Authorization Language in Lean*. 2024. URL: <https://github.com/cedar-policy/cedar-spec>.
- [Bar92] H. Barendregt. «**Lambda Calculi with Types**». In: *Handbook of Logic in Computer Science*. A cura di S. Abramsky, D. M. Gabbay e T. S. E. Maibaum. Vol. 2. Oxford University Press, 1992, pp. 117–309.
- [Car19] M. Carneiro. «**The Type Theory of Lean**». Tesi di laurea mag. Carnegie Mellon University, 2019.
- [CH88] T. Coquand e G. Huet. «**The Calculus of Constructions**». In: *Information and Computation* 76.2–3 (1988), pp. 95–120.
- [Coq86] T. Coquand. «**An Analysis of Girard's Paradox**». In: *Symposium on Logic in Computer Science (LICS '86)*. IEEE, 1986, pp. 227–236.

Bibliografia III

- [Duo26] Y. Duong. *AI Will Be Top of Mind at ICM, Math's Biggest Conference*. Simons Foundation, 4 maggio 2026. 2026. URL: <https://www.simonsfoundation.org/2026/05/04/ai-will-be-top-of-mind-at-icm-maths-biggest-conference/>.
- [Fen+26] T. Feng et al. *Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems*. 2026. arXiv: 2601.22401 [cs.AI]. URL: <https://arxiv.org/abs/2601.22401>.
- [Geu93] H. Geuvers. «Logics and Type Systems». Tesi di dott. Katholieke Universiteit Nijmegen, 1993.
- [Gir72] J.-Y. Girard. «Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur». Thèse d'État. Université Paris VII, 1972.
- [GLT89] J.-Y. Girard, Y. Lafont e P. Taylor. *Proofs and Types*. Vol. 7. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 1989.

Bibliografia IV

- [Gri90] T. G. Griffin. «**A Formulae-as-Types Notion of Control**». In: *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '90)*. ACM, 1990, pp. 47–58.
- [Hof95] M. Hofmann. «**Extensional Concepts in Intensional Type Theory**». Pubblicata da Springer, Distinguished Dissertations, 1997. Tesi di dott. University of Edinburgh, 1995.
- [HS98] M. Hofmann e T. Streicher. «**The Groupoid Interpretation of Type Theory**». In: *Twenty-Five Years of Constructive Type Theory*. Vol. 36. Oxford Logic Guides. Oxford University Press, 1998, pp. 83–111.
- [Hur95] A. J. C. Hurkens. «**A Simplification of Girard's Paradox**». In: *Typed Lambda Calculi and Applications (TLCA '95)*. Vol. 902. LNCS. Springer, 1995, pp. 266–278.
- [Mah26] A. Mahboubi. **Computer-Assisted Mathematics**. Conferenza invitata, TYPES 2026, 4 maggio 2026. 2026.
- [Mar84] P. Martin-Löf. **Intuitionistic Type Theory**. Studies in Proof Theory. Note di Giovanni Sambin delle lezioni di Padova, 1980. Napoli: Bibliopolis, 1984.

- [Mic26] M. Miculan. «**Reliable Decision Loops in Socio-Technical Collective Adaptive Systems**». In: *Leveraging Applications of Formal Methods, Verification and Validation (ISoLA 2026)*. LNCS. Track REoCAS — Rigorous Engineering of Collective Adaptive Systems. In corso di stampa. Springer, 2026.
- [MU21] L. de Moura e S. Ullrich. «**The Lean 4 Theorem Prover and Programming Language**». In: *Automated Deduction (CADE-28)*. Vol. 12699. LNCS. Springer, 2021, pp. 625–635.
- [NPS90] B. Nordström, K. Petersson e J. M. Smith. ***Programming in Martin-Löf's Type Theory: An Introduction***. International Series of Monographs on Computer Science. Oxford University Press, 1990.
- [Pad26] K. Padavic-Callaghan. «**Why Mathematician Terence Tao Thinks AI Must Spark a Rapid Revolution**». In: *New Scientist* (ago. 2026). Intervista a Terence Tao all'ICM di Filadelfia, 6 agosto 2026.

- [Pau93] C. Paulin-Mohring. «**Inductive Definitions in the System Coq: Rules and Properties**». In: *Typed Lambda Calculi and Applications (TLCA '93)*. Vol. 664. LNCS. Springer, 1993, pp. 328–345.
- [Pie02] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [Ren+25] Z. Z. Ren et al. *DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition*. 2025. arXiv: [2504.21801](https://arxiv.org/abs/2504.21801) [cs.CL]. URL: <https://arxiv.org/abs/2504.21801>.
- [Rey84] J. C. Reynolds. «**Polymorphism is not Set-Theoretic**». In: *Semantics of Data Types*. Vol. 173. LNCS. Springer, 1984, pp. 145–156.
- [Sta79] R. Statman. «**Intuitionistic Propositional Logic is Polynomial-Space Complete**». In: *Theoretical Computer Science* 9.1 (1979), pp. 67–72.
- [SU06] M. H. Sørensen e P. Urzyczyn. *Lectures on the Curry–Howard Isomorphism*. Vol. 149. Studies in Logic and the Foundations of Mathematics. Elsevier, 2006.

Bibliografia VII

- [SYA24] P. Song, K. Yang e A. Anandkumar. *Towards Large Language Models as Copilots for Theorem Proving in Lean*. 2024. arXiv: [2404.12534 \[cs.AI\]](#).
- [Tao26a] T. Tao. *Mathematics in the Age of AI*. Saggio basato sulla conferenza pubblica all'ICM 2026. 2026. arXiv: [2608.16753 \[math.HO\]](#).
- [Tao26b] T. Tao. *Terence Tao on AI in Mathematics (and Beyond)*. Sintesi aggiornata delle proprie posizioni; cfr. anche *Mathematical Discourse* e il progetto *First Proof*. 2026. URL: <https://teorth.github.io/tao-web/ai-views.html>.
- [TD+23] T. Tao, Y. Dillies et al. *A Formalisation of the Polynomial Freiman–Ruzsa Conjecture*. Progetto di formalizzazione collaborativa in Lean 4. 2023. URL: <https://github.com/teorth/pfr>.
- [The13] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Princeton: Institute for Advanced Study, 2013. URL: <https://homotopytypetheory.org/book/>.
- [The20] The mathlib Community. «The Lean Mathematical Library». In: *Certified Programs and Proofs (CPP 2020)*. ACM, 2020, pp. 367–381.

Bibliografia VIII

- [The25] The Lean Developers. *The Lean Language Reference*. 2025. URL: <https://lean-lang.org/doc/reference/latest/>.
- [The26] The Leiden Declaration on Artificial Intelligence and Mathematics. *Leiden Declaration on Artificial Intelligence and Mathematics*. Approvata dall'International Mathematical Union; oltre 3000 firmatari. 2026.
- [Wer94] B. Werner. «Une Théorie des Constructions Inductives». Tesi di dott. Université Paris VII, 1994.
- [Wer97] B. Werner. «Sets in Types, Types in Sets». In: *Theoretical Aspects of Computer Software (TACS '97)*. Vol. 1281. LNCS. Springer, 1997, pp. 530–546.
- [Yan+23] K. Yang et al. «LeanDojo: Theorem Proving with Retrieval-Augmented Language Models». In: *Advances in Neural Information Processing Systems (NeurIPS 2023), Datasets and Benchmarks Track*. 2023.